Implementation and Ramifications of TAMER-OP: a Toolkit for Algebraic Module Encodings over $\mathbb{R}^n$ and Other Posets

Erik Mendes Novak

2026-02-22

Contents

1 Introduction **3**

2 Background mathematical information **3**
2.1 Posets as categories; persistence modules as functors . . . 3
2.2 1-parameter persistence . . . 5
2.3 Multiparameter persistence and why it is hard . . . 6
2.4 Two complementary viewpoints . . . 7
2.5 Survey of partial solutions . . . 8
2.6 Brief software landscape . . . 9
2.7 Notation and conventions . . . 9

3 Tame modules **10**
3.1 Why "tame" is the right hypothesis for computation . . . 10
3.2 Encoding posets and encoding maps . . . 11
3.3 Definitions of tameness in this framework . . . 11
3.4 Why non-abelianity fears are usually irrelevant in computation . . . 12
3.5 Fringe presentations as a computable interface . . . 14
3.6 What tameness buys you: mathematical consequences . . . 15
3.7 What tameness buys you: algorithmic consequences . . . 16
3.8 Bridge to implementation . . . 17

4 Overview of TAMER-OP **17**
4.1 Design goals stated mathematically . . . 17
4.2 The end-to-end pipeline . . . 17
4.3 Core mathematical objects in the API . . . 17
4.4 Architectural layering . . . 17
4.5 Reproducibility and correctness . . . 17

5 The presentation formats **17**
5.1 Philosophy: multiple equivalent front-ends . . . 17
5.2 Finite fringe presentations . . . 18
5.3 $\mathbb{Z}^n$ presentations . . . 20
5.4 $\mathbb{R}^n$ and piecewise-linear presentations . . . 21
5.5 Conversions between formats . . . 22
5.6 Validation and other sanity checks . . . 23

6 Finite encoding constructors **24**
6.1 The encoding problem . . . 25
6.2 Region/signature idea . . . 25
6.3 Encoding algorithms by ambient domain . . . 26
6.4 Constructing the finite poset structure . . . 28

6.5 Point location and evaluation map . . . 29
6.6 Building a common encoding for multiple modules . . . 30
6.7 Complexity and failure modes . . . 31
6.8 Correctness and diagnostics . . . 32

7 Invariants **35**
7.1 Meta principle: invariants factor through the encoding . . . 35
7.2 Dimension-type invariants . . . 35
7.3 Rank-type invariants . . . 35
7.4 Mobius inversions and signed measures . . . 35
7.5 Slice-based invariants and distances . . . 35
7.6 Vectorizations and kernels . . . 35
7.7 Homological invariants derived from resolutions . . . 35
7.8 Geometry-aware invariants for when $Q = \mathbb{R}^n$ . . . 35
7.9 Stability and robustness discussion . . . 35
7.10 Practical API summary . . . 35

8 Homological Algebra and Derived Functors **35**
8.1 The Abelian Category of P-modules . . . 35
8.2 Projective and Injective Objects . . . 35
8.3 Resolutions . . . 35
8.4 Ext and Tor: definitions and computational model . . . 35
8.5 Multiplicative structures and higher structure . . . 35
8.6 Double complexes and spectral sequences . . . 35
8.7 The ergonomics of derived categories . . . 35
8.8 Relationship to multigraded commutative algebra . . . 35
8.9 What you use all this for in TDA . . . 35

9 Transport and comparison **35**
9.1 Change of posets as functorial transport . . . 35
9.2 Comparing modules on different encodings . . . 35
9.3 Comparison via invariants . . . 35
9.4 Interoperability with multipers and RIVET . . . 35
9.5 Case studies . . . 35

10 Statistics **35**
10.1 Data model for experiments . . . 35
10.2 Feature extraction recipes . . . 35
10.3 Basic inferential statistics . . . 35
10.4 Learning-oriented workflows . . . 35
10.5 Diagnostic plots . . . 35
10.6 Reproducibility and reporting . . . 35

11 Speed optimizations **35**
11.1 Where time is spent . . . 35
11.2 Tradeoffs in exactness versus speed . . . 35
11.3 Sparsity-aware algorithms . . . 35
11.4 Caching and memoization strategies . . . 35
11.5 Parallelism . . . 35
11.6 Geometry backend: controlling combinatorial explosion . . . 35

12 Benchmarks **35**
12.1 Philosophy and reproducibility . . . 35
12.2 Microbenchmarks (unit tests) . . . 35
12.3 End-to-end benchmarks . . . 35
12.4 Comparisons with multipers and RIVET . . . 35
12.5 Discussion of results . . . 35

1 Introduction

2 Background mathematical information

Persistent homology assigns to data a family of homology groups indexed by a (possibly multi-dimensional) parameter, together with linear maps relating the groups as the parameter increases. The resulting algebraic object is a *persistence module*. In one parameter, persistence modules admit a clean structure theory (barcodes) and robust metrics; in multiple parameters, the same classification program becomes substantially more subtle. This section records the minimal language needed to state the objects studied in this thesis and to motivate the computational strategy pursued by TAMER-OP.

Throughout, fix a coefficient field $\Bbbk$ unless explicitly stated otherwise. When we discuss implementation choices later, we will emphasize why exact arithmetic over $\mathbb{Q}$ is often preferred for certifying ranks and homological invariants (cf. §2.7).

2.1 Posets as categories; persistence modules as functors

Posets and thin categories

Definition 1 (Poset)**.** A *partially ordered set* (poset) is a set Q equipped with a binary relation $\leq$ that is reflexive ($q \leq q$), antisymmetric ($q \leq q'$ and $q' \leq q$ implies $q = q'$), and transitive ($q \leq q'$ and $q' \leq q''$ implies $q \leq q''$).

Definition 2 (Poset as a category)**.** Given a poset $(Q, \leq)$, define a (small) category, which we also denote by Q, whose objects are elements of Q, and such that for $q, q' \in Q$ there is a unique morphism $q \to q'$ if and only if $q \leq q'$, and no morphism otherwise. Composition is forced by transitivity, and identities are forced by reflexivity.

A category arising from a poset in this way is *thin*: between any two objects there is at most one morphism. This viewpoint is the standard bridge between order theory and functorial algebraic constructions.

Modules over a poset

Definition 3 (Poset module / persistence module)**.** Let $(Q, \leq)$ be a poset and let $\mathbf{Vect}_{\Bbbk}$ denote the category of $\Bbbk$-vector spaces and $\Bbbk$-linear maps. A *Q-module* (or *persistence module indexed by Q*) is a covariant functor

$$M \colon Q \longrightarrow \mathbf{Vect}_{\Bbbk}.$$

Concretely, this is the data of:

(i) a vector space $M(q)$ for each $q \in Q$, and

(ii) for each comparable pair $q \leq q'$, a linear *structure map*

$$M(q \leq q') \colon M(q) \to M(q'),$$

subject to the functoriality conditions

$$M(q \leq q) = \mathrm{id}_{M(q)} \qquad \text{and} \qquad M(q \leq q'') = M(q' \leq q'') \circ M(q \leq q') \text{ for } q \leq q' \leq q''.$$

The indexing poset Q will typically be $\mathbb{R}$ or $\mathbb{R}^n$ with the usual order(s), or $\mathbb{Z}^n$ with the product (coordinatewise) order. The degree q is sometimes called a *parameter value*, and $M(q)$ a *fiber* or *stalk*.

Definition 4 (Morphisms)**.** A morphism of Q-modules $f \colon M \to N$ is a natural transformation of functors. Equivalently, it is a family of linear maps $f_q \colon M(q) \to N(q)$ indexed by $q \in Q$ such that for

every $q \le q'$ the following square commutes:

$$\begin{tikzcd} M(q) \arrow[r, "M(q\le q')"] \arrow[d, "f_q"'] & M(q') \arrow[d, "f_{q'}"] \\ N(q) \arrow[r, "N(q\le q')"] & N(q'). \end{tikzcd}$$

We write $\mathrm{Hom}_Q(M,N)$ for the $\Bbbk$-vector space of such natural transformations (when the fibers are finite-dimensional, $\mathrm{Hom}_Q(M,N)$ itself is often computable by linear algebra).

Functor categories are abelian

The category of $\Bbbk$-vector spaces is abelian, and functor categories into an abelian category remain abelian. In particular, Q-modules admit kernels, cokernels, images, and short exact sequences.

Proposition (Pointwise abelian structure)**.** *For any small category $\mathcal{C}$, the functor category $\mathbf{Vect}_\Bbbk^{\mathcal{C}} = \mathrm{Fun}(\mathcal{C}, \mathbf{Vect}_\Bbbk)$ is abelian. In particular, for $\mathcal{C} = Q$ a poset category, kernels and cokernels are computed* objectwise*: for a morphism $f\colon M \to N$ one has*

$$(\ker f)(q) = \ker(f_q), \qquad (\operatorname{coker} f)(q) = \operatorname{coker}(f_q),$$

and similarly for images and coimages, with the induced structure maps coming from functoriality.

Remark. Proposition 2.1 is conceptually important because it reduces many categorical constructions on persistence modules to familiar linear algebra at each parameter value. Much of TAMER-OP is built around exploiting this pointwise nature after reducing an a priori infinite indexing poset to a finite one.

Upsets, downsets, and indicator modules

A recurrent theme (central to Miller's theory of tameness) is that many computationally tractable persistence modules can be assembled from very simple "indicator" modules associated to order ideals and filters.

Definition 5 (Upsets and downsets)**.** Let Q be a poset.

(i) An *upset* (order filter) is a subset $U \subseteq Q$ such that if $q \in U$ and $q \le q'$, then $q' \in U$.

(ii) A *downset* (order ideal) is a subset $D \subseteq Q$ such that if $q' \in D$ and $q \le q'$, then $q \in D$.

For $q \in Q$, the *principal upset* and *principal downset* are

$$\uparrow q := \{q' \in Q : q \le q'\}, \qquad \downarrow q := \{q' \in Q : q' \le q\}.$$

Definition 6 (Indicator modules)**.** Let $U \subseteq Q$ be an upset. The *upset indicator module* $\Bbbk\{U\}$ is the Q-module defined by

$$\Bbbk\{U\}(q) := \begin{cases} \Bbbk & q \in U, \\ 0 & q \notin U, \end{cases}$$

and for $q \le q'$ the structure map $\Bbbk\{U\}(q \le q')$ is the identity $\Bbbk \to \Bbbk$ when $q, q' \in U$ and is the unique zero map otherwise. Likewise, for a downset $D \subseteq Q$, define the *downset indicator module* $\Bbbk\{D\}$ by the same formula with D in place of U.

Remark. The closure property in Definition 25 is exactly what guarantees functoriality for the "identity within the region, zero outside" convention in Definition 26. For general subsets $S \subseteq Q$, the same prescription can fail to define a functor because composition along chains may force a map to be zero even when both endpoints lie in S.

Remark (Yoneda intuition). When Q is viewed as a category, the representable functor $\mathrm{Hom}_Q(q,-)$ takes values in sets; its $\Bbbk$-linearization yields precisely the principal upset indicator $\Bbbk[\uparrow q]$. Dually, the corepresentable $\mathrm{Hom}_Q(-,q)$ linearizes to $\Bbbk[\downarrow q]$. For finite Q, these principal indicator modules behave like indecomposable projectives and injectives in the functor category, and they serve as building blocks for resolutions (compare with [Mil20]).

2.2 1-parameter persistence

This subsection recalls the classical one-parameter story, both to establish notation and to emphasize what breaks in the multiparameter setting.

From filtrations to persistence modules

Definition 7 (Filtration)**.** Let Q be a poset. A *Q-filtration* of a topological space X is a functor

$$\mathcal{F}\colon Q \longrightarrow \mathbf{Top}$$

such that $\mathcal{F}(q)$ is a subspace of X for each q, and for $q \leq q'$ the morphism $\mathcal{F}(q \leq q')$ is the inclusion $\mathcal{F}(q) \hookrightarrow \mathcal{F}(q')$. Equivalently, $\mathcal{F}$ is an order-preserving assignment of subspaces.

Applying singular homology with coefficients in $\Bbbk$ yields a persistence module.

Definition 8 (Persistent homology module)**.** Given a Q-filtration $\mathcal{F}$ and an integer $i \geq 0$, the associated ith *persistent homology module* is the composite functor

$$H_i(\mathcal{F};\Bbbk)\colon Q \xrightarrow{\mathcal{F}} \mathbf{Top} \xrightarrow{H_i(-;\Bbbk)} \mathbf{Vect}_{\Bbbk}.$$

The case $Q = (\mathbb{R}, \leq)$ (or $Q = \mathbb{Z}$) is the classical *one-parameter* setting. The filtration may arise, for example, from sublevel sets of a function $f\colon X \to \mathbb{R}$, where $\mathcal{F}(t) = f^{-1}((-\infty, t])$.

Interval modules and barcodes

The exceptional feature of one-parameter persistence is that (under mild finiteness hypotheses) modules admit a complete decomposition into interval summands.

Definition 9 (Interval module)**.** Let $I \subseteq \mathbb{R}$ be an interval (possibly infinite or half-open, depending on the chosen convention). The *interval module* $\Bbbk_I$ is the $\mathbb{R}$-module defined by

$$\Bbbk_I(t) = \begin{cases} \Bbbk & t \in I, \\ 0 & t \notin I, \end{cases}$$

with structure maps $\Bbbk_I(s \leq t)$ equal to the identity on $\Bbbk$ when $s, t \in I$ and the zero map otherwise.

To state the classification theorem, one usually imposes a finiteness condition, such as *pointwise finite-dimensional* (p.f.d.), meaning $\dim_{\Bbbk} M(t) < \infty$ for all t, together with an appropriate tameness condition to rule out pathologies in the indexing by $\mathbb{R}$ (see [CB15, CDSGO16]).

Theorem 10 (Barcode decomposition, informal statement)**.** *Let M be a p.f.d. persistence module indexed by $\mathbb{R}$ satisfying a standard tameness condition (e.g. q-tameness). Then M decomposes as a direct sum of interval modules,*

$$M \cong \bigoplus_{\alpha \in A} \Bbbk_{I_\alpha},$$

and the multiset of intervals $\{I_\alpha\}$ is uniquely determined by M up to reordering. This multiset is the barcode *(or persistence diagram) of M.*

The earliest formulations of this result for discrete indexing and finitely generated modules go back to persistent homology in computational topology; see, for example, [ZC05, EH10]. For $\mathbb{R}$-indexed modules, a definitive representation-theoretic proof is given by Crawley-Boevey [CB15].

Stability and distances

A major reason barcodes are useful in applications is *stability*: small changes in input data yield small changes in the output barcode with respect to a natural metric. There are several equivalent (or closely related) metrics in one parameter, notably the bottleneck distance on persistence diagrams and the interleaving distance on persistence modules; see [CDSGO16, Les15] for systematic treatments.

For the purposes of this thesis, we will use one-parameter stability primarily as a benchmark: it is an example of a situation where both (i) a complete discrete invariant exists (Theorem 10) and (ii) that invariant interacts well with perturbations. Both features become substantially more complicated in multiple parameters.

2.3 Multiparameter persistence and why it is hard

Definition and basic examples

Definition 11 (Multiparameter persistence module)**.** Fix $n \geq 2$. A (real-parameter) *n-parameter persistence module* is a $\mathbb{R}^n$-module, i.e. a functor $M\colon (\mathbb{R}^n, \leq) \to \mathbf{Vect}_{\Bbbk}$, where the order is coordinatewise:

$$a = (a_1, \ldots, a_n) \leq b = (b_1, \ldots, b_n) \quad \Longleftrightarrow \quad a_i \leq b_i \text{ for all } i.$$

Likewise, a *lattice* n-parameter persistence module is a $\mathbb{Z}^n$-module.

Such modules arise from *multifiltrations* $\mathcal{F}\colon \mathbb{R}^n \to \mathbf{Top}$, for instance from a function $f\colon X \to \mathbb{R}^n$ via $\mathcal{F}(a) = f^{-1}((-\infty, a_1] \times \cdots \times (-\infty, a_n])$, and then applying homology as in Definition 8.

Why barcodes do not generalize directly

When $n \geq 2$, the classification landscape changes qualitatively. There is no analog of Theorem 10 in which modules decompose uniquely into interval-like building blocks (rectangles, axis-aligned boxes, etc.). One reason is representation-theoretic: the category of multiparameter persistence modules (even with finite-dimensionality restrictions) contains families of indecomposables depending on continuous parameters and exhibits behavior described as "wild" in the sense of quiver representation theory [CZ09, Oud15].

At an informal level, "wild" means that the isomorphism classification problem is at least as complicated as classifying representations of arbitrary finite-dimensional algebras. Consequently, one does not expect a complete discrete invariant such as a barcode to exist in general.

Remark (What replaces barcodes?). In multiparameter persistence, one typically trades completeness for computability and stability: instead of a single complete invariant, one studies families of computable invariants (rank-based, homological, and/or slice-based) that are informative for specific tasks and data regimes. Much of the design philosophy of TAMER-OP is to build a flexible pipeline that supports such families in a uniform algebraic framework.

Invariants that still make sense

Even though complete classification fails, many useful invariants remain available.

Definition 12 (Rank invariant)**.** Let M be a Q-module. The *rank invariant* of M is the function

$$\rho_M\colon \{(q, q') \in Q \times Q : q \leq q'\} \longrightarrow \mathbb{Z}_{\geq 0}, \qquad \rho_M(q, q') := \operatorname{rank}\big(M(q \leq q')\big).$$

Definition 13 (Hilbert function / dimension function)**.** The *dimension function* (often called the *Hilbert function* in graded settings) of a Q-module M is

$$h_M\colon Q \to \mathbb{Z}_{\geq 0}, \qquad h_M(q) := \dim_{\Bbbk} M(q).$$

Remark. For $Q = \mathbb{N}^n$ and suitably finite modules, h_M is literally the Hilbert function of a multigraded module over a polynomial ring. For general posets, it is still a basic and often stable summary of size.

Another robust idea is to reduce to one parameter by restricting along monotone maps from a totally ordered set into Q.

Definition 14 (Restriction to a chain / slice)**.** Let C be a totally ordered set (often $C = \mathbb{R}$ or $\mathbb{Z}$) and let $\gamma\colon C \to Q$ be an order-preserving map. The *restriction* (or *slice*) of a Q-module M along γ is the C-module $\gamma^* M$ defined by composition:

$$\gamma^* M := M \circ \gamma\colon C \to \mathbf{Vect}_{\Bbbk}.$$

When C is totally ordered and γ is injective, the image of γ is a chain in Q.

The restrictions $\gamma^* M$ are one-parameter persistence modules, hence admit barcodes under the usual hypotheses. Slicing therefore packages multiparameter data into a family of one-parameter summaries; this idea is central in 2-parameter visualization tools such as RIVET [RIVET].

Finally, there are homological invariants arising from resolutions and derived functors (Ext, Tor) that encode multi-parameter interactions. These will be pursued systematically later in the thesis, but the guiding idea is already visible in Miller's program: introduce a finiteness condition strong enough to guarantee finite presentations and resolutions (and thus computable derived invariants), while weak enough to encompass common persistence modules arising from data [Mil20].

2.4 Two complementary viewpoints

Multiparameter persistence can be approached from at least two complementary perspectives. The computational architecture of TAMER-OP is explicitly designed to move between them.

Topological viewpoint: multifiltrations

At the source is topology: data are used to build a multifiltration $\mathcal{F}\colon \mathbb{R}^n \to \mathbf{Top}$ (or $\mathbb{Z}^n \to \mathbf{Top}$), and persistent homology applies a functor $H_i(-;\Bbbk)$ to obtain

$$M_i := H_i(\mathcal{F};\Bbbk)\colon \mathbb{R}^n \to \mathbf{Vect}_{\Bbbk}.$$

The topological viewpoint is advantageous for:

- motivating what invariants should mean geometrically (birth, death, and interactions across parameters);
- importing stability statements from the theory of interleavings and perturbation bounds; and
- making contact with geometric constructions of filtrations (sublevel sets, distance functions, Vietoris–Rips and Čech complexes, etc.; see [EH10]).

Algebraic viewpoint: multigraded commutative algebra

When the indexing poset is discrete and additive, persistence modules are closely related to multigraded modules over polynomial rings.

Proposition (Equivalence with multigraded modules (discrete setting))**.** *Let $n \geq 1$ and consider the poset $(\mathbb{N}^n, \leq)$ with coordinatewise order. Then the category of $\mathbb{N}^n$-modules $M\colon \mathbb{N}^n \to \mathbf{Vect}_{\Bbbk}$ is equivalent to the category of $\mathbb{N}^n$-graded modules over the polynomial ring $\Bbbk[x_1, \ldots, x_n]$ with degree-preserving homomorphisms.*

Remark (Sketch of correspondence). Given a persistence module M, form the graded $\Bbbk$-vector space $\bigoplus_{a\in\mathbb{N}^n} M(a)$ and let x_i act by the structure maps $M(a \leq a + e_i)\colon M(a) \to M(a + e_i)$. Functoriality ensures that the x_i commute. Conversely, a graded $\Bbbk[x_1, \ldots, x_n]$-module determines the structure maps by multiplication with monomials. See [CZ09, Oud15] for detailed treatments in the persistence context.

This algebraic viewpoint brings powerful tools—resolutions, Betti numbers, Hilbert series—but also reveals why classification becomes hard when $n \geq 2$: $\Bbbk[x_1, \ldots, x_n]$ is no longer a PID, and multigraded modules can have intricate syzygies and moduli.

Finite posets and incidence algebras

TAMER-OP often works with finite posets (encoding posets) obtained by compressing a large or continuous parameter space. For a finite poset P, P-modules can be viewed as finite-dimensional representations of the incidence algebra of P (or of a quiver with commutativity relations determined by P). This provides both:

- a representation-theoretic interpretation of projectives/injectives and resolutions (cf. Remark 2.1), and
- an algorithmic opportunity: for finite P, most constructions reduce to finite linear algebra, and many "global" constraints can be enforced by working only along cover relations (Hasse edges).

2.5 Survey of partial solutions

Because multiparameter persistence resists complete classification, a large body of work focuses on *partial* invariants and practical pipelines. The following four themes recur throughout the literature and motivate the design choices of this thesis.

(i) Slicing and fibered barcodes

The restriction construction in Definition 14 reduces a multiparameter module to a one-parameter module, hence to a barcode. Varying the slice (for instance, varying a line of positive slope in $\mathbb{R}^2$) yields a family of barcodes that can be used for visualization or for defining distances by integrating (or taking suprema of) one-parameter distances. In the 2-parameter case, the *fibered barcode* viewpoint underlies the RIVET framework [RIVET].

From a computational standpoint, slicing has two key advantages:

(i) it leverages mature one-parameter algorithms, and

(ii) it provides stable summaries for applications even when the full module is too large to manipulate directly.

Its main limitation is incompleteness: different modules can have identical slices.

(ii) Rank-based invariants and Möbius inversion

The rank invariant (Definition 12) is one of the most basic multiparameter invariants. In one parameter, the rank function determines the barcode, and the barcode can be recovered from ranks by an inclusion–exclusion (Möbius inversion) formula. In higher parameters, an analogous inversion yields *signed* measures on axis-aligned rectangles (sometimes called *signed barcodes* or *rectangle measures*). These objects can be used as:

- sparse, computable summaries of the rank invariant on a chosen grid, and
- feature representations for statistics/learning pipelines (e.g. kernels on signed measures).

From the algebraic side, such Möbius inversions are naturally phrased on products of chains, and they fit well with finite encodings (cf. (iv) below).

(iii) Commutative algebra invariants via resolutions

Under the equivalence in Proposition 2.4, a $\mathbb{N}^n$-module becomes a multigraded $\Bbbk[x_1, \ldots, x_n]$-module, and one can study:

- minimal free resolutions and multigraded Betti numbers,
- Tor/Ext groups and their algebra structures,
- associated primes and socles, and
- Hilbert series and local cohomology.

These invariants detect multi-parameter interactions that rank functions and slices can miss. However, they require a finiteness condition strong enough to ensure that resolutions are finite in a computational sense. For real-parameter modules, the usual noetherian hypotheses fail, motivating Miller's theory of *tameness* as a replacement [Mil20].

(iv) Geometric/combinatorial encoding strategies

A guiding principle in this thesis is that many modules of interest are *piecewise constant* over the parameter space in a controlled way. An *encoding* compresses the indexing poset Q (often $\mathbb{R}^n$ or $\mathbb{Z}^n$) to a finite poset P so that the module factors through P and all subsequent computations may be carried out on the finite object.

In Miller's framework, tameness can be characterized in several equivalent ways, including (among others) the existence of a finite encoding and the existence of certain finite "fringe" presentations by upset and

downset indicator modules [Mil20]. The computational moral is that once such a finite encoding is constructed, one can safely deploy the abelian and homological algebra of finite poset modules (kernels, images, Ext/Tor, resolutions) using finite linear algebra.

2.6 Brief software landscape

This thesis is not primarily a software survey, but it is useful to clarify how different toolchains emphasize different parts of the multiparameter landscape. We will return to more detailed comparisons in later sections, after the TAMER-OP data model and pipeline are established.

- **RIVET.** In the 2-parameter case, RIVET is centered on the fibered barcode: it supports interactive exploration of how one-parameter barcodes vary over a family of lines, and it provides infrastructure for rank-based visualization and related distances [RIVET].
- **multipers.** The multipers ecosystem (as commonly used in applied TDA) targets scalable computation of multiparameter summaries and feature maps (often slice-based, rank-based, or image/landscape-type constructions). Its emphasis is typically on practical pipelines rather than complete algebraic structure. (A precise feature-by-feature comparison depends on the chosen backends and will be addressed later.)
- **TAMER-OP.** TAMER-OP is designed around Miller's tame-module viewpoint: build or ingest a presentation over a large indexing poset, compress it via a finite encoding poset, and then compute a broad family of invariants (rank-based, homological/derived, geometric, and statistical) by systematically exploiting abelian and homological algebra over the finite encoding.

Remark (Setting expectations for comparisons). Comparisons are most meaningful when the underlying mathematical object is held fixed. For example, comparing fibered barcodes is natural when both toolchains work with 2-parameter modules and slice-based summaries. Comparing resolution-based invariants is meaningful when both toolchains expose (or can emulate) multigraded homological algebra. TAMER-OP's main distinguishing feature is that it treats *finite encodings* as first-class objects, so that many invariants can be computed uniformly once the encoding is in place.

2.7 Notation and conventions

Coefficients and exactness

Unless stated otherwise, modules take values in $\mathbf{Vect}_{\Bbbk}$ for a fixed field $\Bbbk$. In implementations, there is a persistent tradeoff between speed and certifiability:

- Over floating-point fields (e.g. $\mathbb{R}$), numerical linear algebra is fast but can introduce rank errors near degeneracies.
- Over exact fields (notably $\mathbb{Q}$), ranks, kernels, and Smith/row-reduction type computations are certifiable but may be more expensive.

TAMER-OP is designed so that exact computations over $\mathbb{Q}$ can be used when needed, especially for invariants derived from ranks and homological algebra (kernels, cokernels, Ext/Tor dimensions).

Orders on $\mathbb{Z}^n$ and $\mathbb{R}^n$

We use the coordinatewise order on $\mathbb{Z}^n$ and $\mathbb{R}^n$:

$$a \leq b \Leftrightarrow a_i \leq b_i \text{ for all } i.$$

When writing $a < b$, we mean $a \leq b$ and $a \neq b$. The standard basis vectors are $e_1, \ldots, e_n$.

For a finite poset P, we generally denote its order by $\leq$ as well; the intended meaning should be clear from context. When P is used as an encoding poset for an ambient parameter space, elements of P are sometimes called *regions*.

Birth/death terminology and (co)presentations

When working with indicator modules (Definition 26) and their linear combinations, we follow the "birth/death" intuition from persistence:

- Upsets model *birth* regions (where generators become present and persist upward in the order).
- Downsets model *death* constraints (where relations become enforced downward in the order).

A *presentation* will typically mean a cokernel of a map between free objects built from upset indicators, while a *copresentation* dually involves downset indicators. In Miller's terminology, *fringe presentations* mix these two directions and are tailored to tame modules over general posets [Mil20].

3 Tame modules

Multiparameter persistence is often presented as a story about functoriality: a multifiltration of spaces (or chain complexes) gives rise to a functor from a parameter poset into vector spaces, and this functor is the algebraic object one studies. The difficulty is that, once the parameter poset has dimension ≥ 2, there is no reasonable discrete classification of all such functors, even under strong finiteness conditions such as pointwise finite-dimensionality; in representation-theoretic language, the classification problem becomes "wild" in general [CZ09]. For computational purposes one therefore needs a *hypothesis of controlled complexity*—a notion of finiteness that is strong enough to guarantee that modules admit finite descriptions and are stable under the constructions one cares about, but flexible enough to encompass the examples arising in applications.

In this thesis we adopt Ezra Miller's framework of *tame modules over posets* [Mil20]. The guiding idea is that the module should have only finitely many distinct "regions of algebraic behavior" in parameter space, so that it can be represented by *finite data* after passing through a suitable *encoding*. A primary goal of TAMER-OP is to turn this philosophy into concrete data structures and algorithms; the present section sets up the mathematical backbone that makes such an implementation sensible.

3.1 Why "tame" is the right hypothesis for computation

The classical 1-parameter theory enjoys a powerful structure theorem: under mild hypotheses, persistence modules decompose as direct sums of interval modules, and the resulting multiset of intervals is encoded by a barcode or persistence diagram. This viewpoint underlies much of applied persistent homology, as well as practical algorithms for computing invariants from filtered data [ZC05]. One rigorous formulation of this classification is due to Crawley–Boevey: pointwise finite-dimensional persistence modules over a totally ordered index set admit interval decompositions under appropriate finiteness hypotheses [CB15].

The multiparameter setting behaves fundamentally differently. Even for $\mathbb{Z}^n$-indexed modules with $n \geq 2$, indecomposables occur in families and the category does not admit a discrete complete invariant analogous to a barcode [CZ09]. From a computational perspective, this means that one should not expect a *complete* classification of multiparameter persistence modules by a finite combinatorial object. Instead, one aims for a framework in which

(i) the objects of interest admit *finite representations*,

(ii) the relevant algebraic constructions (kernels/cokernels, derived functors, etc.) stay within the class, and

(iii) the invariants one computes are *functorial* and stable under natural operations on the input data.

Miller's notion of tameness is designed with exactly these desiderata in mind. Very roughly, a module is tame if it is constant on finitely many regions of parameter space and therefore can be expressed via a finite "model poset" that records the comparability relations among these regions [Mil20]. The definition is arranged so that the category of tame modules (with an appropriate restriction on morphisms) is abelian, hence supports homological algebra; moreover, tameness admits several equivalent characterizations—combinatorial, topological, and homological—that are captured by the syzygy theorem (Theorem 31 below).

3.2 Encoding posets and encoding maps

The central finiteness notion used throughout this thesis is *finite poset encoding*. Intuitively, an encoding replaces a large (typically infinite) poset Q by a smaller poset P whose elements represent "regions" of the parameter space, such that the module on Q is obtained by pullback from a module on P.

Definition 15 (Poset encoding [Mil20, Def. 4.1])**.** Let Q be a poset, viewed as a category in the usual way. An *encoding* of a Q-module M by a poset P consists of

(i) an order-preserving map (equivalently, a functor) $\pi\colon Q \to P$, and

(ii) a P-module $H\colon P \to \mathbf{Vect}_{\Bbbk}$

together with an isomorphism of Q-modules

$$M \;\cong\; \pi^* H \;:=\; H \circ \pi.$$

The encoding is called *finite* if P has finitely many elements and each stalk $H(p)$ is a finite-dimensional $\Bbbk$-vector space.

Remark (Encodings as "finite models"). If $M \cong \pi^* H$, then M is constant on each fiber $\pi^{-1}(p)$: for any $q, q' \in Q$ with $\pi(q) = \pi(q') = p$, the only morphism $p \to p$ in the poset category is the identity, so the structure maps of M restrict to identities on the shared stalk $H(p)$. Thus a finite encoding exhibits M as having finitely many "constant regions" indexed by P. Conversely, in many geometric situations a partition of the parameter space into finitely many constant regions determines a finite encoding by taking P to be the poset of regions ordered by reachability (or by closure relations); see [Mil20] for the general equivalence among the various constructions.

The functorial viewpoint is especially important in what follows, so we record the basic formalism.

Definition 16 (Pullback along a poset morphism)**.** Let $\pi\colon Q \to P$ be a poset morphism. Precomposition defines a functor

$$\pi^*\colon \mathbf{Vect}_{\Bbbk}^P \longrightarrow \mathbf{Vect}_{\Bbbk}^Q, \qquad H \longmapsto H \circ \pi.$$

We refer to π^* as the *pullback* (or restriction) functor along π.

Lemma 17 (Exactness of pullback)**.** *For any poset morphism $\pi\colon Q \to P$, the pullback functor π^* is exact. In particular, it preserves finite limits and colimits (kernels, cokernels, and finite direct sums).*

Proof. The category $\mathbf{Vect}_{\Bbbk}^P$ of P-modules is a functor category with values in $\mathbf{Vect}_{\Bbbk}$, hence limits and colimits are computed pointwise. Given a short exact sequence $0 \to H' \to H \to H'' \to 0$ of P-modules, evaluating at any $p \in P$ yields a short exact sequence of vector spaces. Precomposing with π and then evaluating at $q \in Q$ simply evaluates the original sequence at $\pi(q)$, so exactness is preserved at every stalk, hence globally. □

Definition 15 formalizes the idea that the module "factors through" a finite poset. The following terminology, also standard in [Mil20], is useful when one wishes to emphasize the dependence on a fixed encoding map.

Definition 18 (Subordinate encoding [Mil20, Def. 4.7])**.** An encoding map $\pi\colon Q \to P$ is said to be *subordinate* to a Q-module M if there exists a P-module H such that $M \cong \pi^* H$.

Example 19 (Indicator modules admit trivial finite encodings [Mil20, Ex. 4.6])**.** Let $U \subseteq Q$ be an upset (upward-closed set). Then the indicator module $\Bbbk\{U\}$ (defined in Subsection 3.5) admits a finite encoding by the two-element chain $0 < 1$ via the map $\pi\colon Q \to \{0 < 1\}$ sending U to 1 and $Q \setminus U$ to 0; the corresponding module H has $H(1) = \Bbbk$ and $H(0) = 0$. Dually, any downset indicator module $\Bbbk[D]$ is encoded by the same chain with the roles of 0 and 1 reversed.

3.3 Definitions of tameness in this framework

The previous subsection defined what it means for a module to be encoded by a fixed poset P. Tameness is the assertion that such an encoding exists with P finite.

Definition 20 (P-tame and tame modules)**.** Fix a poset Q.

(i) Let P be a finite poset. A Q-module M is *P-tame* if there exists a poset morphism $\pi\colon Q \to P$ and a P-module H with finite-dimensional stalks such that $M \cong \pi^* H$.

(ii) A Q-module M is *tame* if it is P-tame for some finite poset P. Equivalently, M admits a finite poset encoding subordinate to it in the sense of Definition 15.

Remark (Comparison with other finiteness notions). There are several finiteness conditions in the persistence literature, especially over $\mathbb{R}^n$ and $\mathbb{Z}^n$, including variants of "constructibility" and "q-tameness" based on finiteness of ranks of structure maps. Those conditions are well adapted to stability questions and to certain metric structures, but they do not by themselves supply a finite combinatorial model that supports the full suite of homological computations one might wish to perform. Finite encodings are a *stronger, more combinatorial* hypothesis: they guarantee that all information about M is carried by a finite poset P together with finite linear algebra on the stalks of a P-module. In particular, tameness is expressly designed to enable algorithmic computation in principle [Mil20].

A subtle point is that *not every* Q-module homomorphism between tame modules should be regarded as "computably tame". Allowing all homomorphisms can take one outside the tame world (for example, kernels of arbitrary maps need not be finitely encoded). The correct remedy is to restrict to morphisms compatible with a common finite description.

Definition 21 (Tame morphisms and the tame category [Mil20, Def. 4.27, Def. 4.29])**.** Let M, N be tame Q-modules. A homomorphism $\varphi\colon M \to N$ of Q-modules is called *tame* if there exists a finite "constant subdivision" of Q that is subordinate to both M and N, such that on each constant region the induced linear map is independent of the chosen parameter value (see [Mil20, Def. 4.27] for the precise formulation). The *category of tame Q-modules* is the category whose objects are tame Q-modules and whose morphisms are tame homomorphisms.

Theorem 22 (Tame modules form an abelian category [Mil20, Prop. 4.31])**.** *For any poset Q, the category of tame Q-modules (with tame morphisms) is abelian. In particular, it has kernels and cokernels, finite products and coproducts, and every monomorphism (resp. epimorphism) is a kernel (resp. cokernel).*

3.4 Why non-abelianity fears are usually irrelevant in computation

The worry voiced by many users of multiparameter persistence is the following. The ambient category $\mathbf{Vect}_{\Bbbk}^{Q}$ of Q-modules is abelian, so it is natural to form kernels, cokernels, homology, and derived functors there. However, the full subcollection of *tame* objects inside $\mathbf{Vect}_{\Bbbk}^{Q}$ is not automatically stable under these operations if one allows *arbitrary* natural transformations between tame objects. In other words, tameness can fail to be preserved if one performs abelian constructions in $\mathbf{Vect}_{\Bbbk}^{Q}$ without restricting the class of morphisms.

This subsection unpacks the mathematical content behind that concern and explains why, from the standpoint of computation, the bad behavior is almost never encountered. The key point is that there are two distinct notions of "morphism between tame modules", and only one of them is compatible with finite data structures.

Homomorphisms of tame objects versus tame morphisms

Recall from Remark **??** that a finite encoding $\pi\colon Q \to P$ converts a tame Q-module into a P-module for a finite poset P. In that finite setting, morphisms are ordinary natural transformations in $\mathbf{Vect}_{\Bbbk}^{P}$, hence are controlled by finitely many linear maps.

Inside $\mathbf{Vect}_{\Bbbk}^{Q}$, however, a natural transformation $\varphi\colon M \to N$ between tame modules can vary on infinitely many degrees even when M and N are each constant on finitely many regions. Miller isolates the maps that are compatible with finite encodings by requiring *constancy of the transformation on constant regions*: a homomorphism φ is called *tame* if there exists a finite constant subdivision subordinate to both M and N such that on each region I the composite map $M_I \to M_q \to N_q \to N_I$ is independent of $q \in I$ [Mil20, Def. 4.27]. The category of tame modules uses these tame homomorphisms as its morphisms [Mil20, Def. 4.29 and Rem. 4.30]. The resulting category is abelian by Theorem 22.

A concrete failure mode

The distinction between arbitrary homomorphisms and tame morphisms is not cosmetic. The following example demonstrates that kernels can leave the tame world if the morphism is allowed to vary infinitely within a constant region.

Example 23 (A kernel of a homomorphism of tame modules need not be tame [Mil20, Ex. 4.25])**.** Let $Q = \mathbb{R}^2$ with the coordinatewise partial order, and let $L = \{(x, y) \in \mathbb{R}^2 : x + y = 1\}$ be the antidiagonal line. Let $U \subseteq Q$ be the closed upset above L, and let U° be its interior. Consider the tame modules

$$M = \Bbbk\{U\} \oplus \Bbbk\{U\}, \qquad N = \Bbbk\{U\}/\Bbbk[U^\circ],$$

where N is supported on L. Define a surjection $\varphi\colon M \twoheadrightarrow N$ by taking φ to be zero on U° and, for each point $(x, y) \in L$, sending the two standard basis vectors of $M_{(x,y)} = \Bbbk^2$ to the scalars y and $-x$ in $N_{(x,y)} = \Bbbk$.

Then $K = \ker \varphi$ is *not* tame. Indeed, for $(x, y) \in L$, the stalk $K_{(x,y)}$ is the one-dimensional subspace of $\Bbbk^2$ spanned by (x, y), and these lines are pairwise distinct as (x, y) varies along L. Consequently, any constant subdivision subordinate to K must separate distinct points of L into distinct constant regions, forcing infinitely many regions.

The phenomenon in Example 23 is that M and N are each constant on a finite subdivision of Q (three regions: below L, on L, and above L), but the map φ is *not* constant on the region L itself. Hence φ is not a tame homomorphism, so there is no reason for its kernel to remain tame.

The tame category is closed under the usual constructions

Once morphisms are required to be tame, the basic abelian operations are safe.

Lemma 24 (Kernels and cokernels of tame homomorphisms are tame [Mil20, Lem. 4.28])**.** *Let $\varphi\colon M \to N$ be a tame homomorphism of Q-modules. Then $\ker \varphi$ and $\operatorname{coker} \varphi$ are tame modules, and the canonical maps $\ker \varphi \to M$ and $N \to \operatorname{coker} \varphi$ are tame morphisms. The same statement holds in the semialgebraic, PL, subanalytic, or class X settings when the constant subdivision witnessing tameness is of the corresponding type.*

Proof. Choose a finite constant subdivision of Q that is subordinate to M, N, and φ (as required in the definition of a tame homomorphism). For each constant region I, fix identifications $M_q \cong M_I$ and $N_q \cong N_I$ for all $q \in I$ so that the structure maps of M and N agree with the transition maps among the regionwise vector spaces. By tameness of φ, the induced linear map $M_I \to N_I$ is independent of $q \in I$.

Define regionwise vector spaces

$$K_I := \ker(M_I \to N_I), \qquad C_I := \operatorname{coker}(M_I \to N_I).$$

Using the structure maps between regions, these assemble into Q-modules K and C. By construction, $K = \ker \varphi$ and $C = \operatorname{coker} \varphi$ in the abelian category $\mathbf{Vect}_\Bbbk^Q$, and the chosen constant subdivision is subordinate to both K and C. Hence K and C are tame. The auxiliary hypotheses (PL, semialgebraic, etc.) are preserved because the same subdivision witnesses tameness in those settings. □

Lemma 24 is the formal version of the slogan that "computations with tame modules stay tame" as long as the maps used in the computation come from finite data. Example 23 shows that if one insists on allowing *all* homomorphisms between tame objects, then kernels and cokernels can escape the tame world, but those escaping morphisms are precisely the ones that do not admit a finite encoding.

Why the pathological maps do not appear in algorithms

Any explicit computation in this thesis ultimately proceeds by passing to some finite encoding $\pi\colon Q \to P$ and working in the finite functor category $\mathbf{Vect}_\Bbbk^P$. In that regime, every morphism is represented by finitely many linear maps, and pulling back along π produces a tame homomorphism in the sense of Definition 21. Exactness of pullback (Lemma 17) then implies that kernels and cokernels computed at the finite level remain finite after pulling back.

More explicitly, if $M \cong \pi^* H$ and $N \cong \pi^* G$ for P-modules H and G, and if $\bar{\varphi}\colon H \to G$ is a morphism in $\mathbf{Vect}_{\Bbbk}^{P}$, then $\varphi := \pi^* \bar{\varphi}$ is a tame homomorphism of Q-modules, and

$$\ker(\varphi) \cong \pi^*(\ker \bar{\varphi}), \qquad \operatorname{coker}(\varphi) \cong \pi^*(\operatorname{coker} \bar{\varphi}),$$

because π^* is exact. Therefore the outputs of the standard constructions (kernel, cokernel, homology of a complex, and derived functors computed via finite resolutions) remain tame whenever the computation is expressed at the level of a finite encoding. In particular, the "risky" behavior in Example 23 cannot occur unless one is willing to specify a morphism that varies on infinitely many degrees inside a constant region, which is not compatible with the finite data structures used for encoding and computation.

Recent results that restore full abelianity under geometric hypotheses

The preceding discussion explains why Miller restricts morphisms in the tame category. There is also a complementary line of results showing that, for many geometric classes of finite encodings, the restriction is in practice unnecessary: under suitable hypotheses on encoding fibers, every homomorphism between encodable modules can be realized inside a finite encoding.

One precise formulation is due to Waas [Waa24], who introduces a connectivity condition on encoding fibers ensuring that any finite set of encodable modules admits a *common* encoding that captures *all* morphisms between them [Waa24, Prop. 3.5]. In particular, the resulting class of encodable persistence modules forms a full abelian subcategory of $\mathbf{Vect}_{\Bbbk}^{Q}$ [Waa24, Thm. 1.1]. For parameter posets $Q = \mathbb{R}^n$, his hypotheses are satisfied by natural families of "geometric" regions built from topologically closed PL, semialgebraic, or subanalytic upsets (and their complements and finite unions).

Conceptually, the point is that the failure in Example 23 relies on large disconnected families of degrees (here the continuum of points on L) along which a map can vary independently. Connectivity hypotheses on encoding fibers rule out that degree of freedom, forcing morphisms to be controlled by finite combinatorial data. Waas also notes that one can alternatively prove the corresponding subanalytic statement by translating the problem into constructible sheaf theory and invoking results of Miller and Kashiwara-Schapira [Waa24, Rem. 3.19], [Mil23, KS18].

These developments give a precise mathematical interpretation of the informal claim that "if the input is tame, then the output is almost always tame": the non-tame outputs arise from morphisms that do not admit finite encodings, and such morphisms are excluded either (i) by working in the tame category (tame morphisms by definition), or (ii) by restricting attention to the geometric subclasses of tame modules for which all relevant morphisms are automatically encodable.

3.5 Fringe presentations as a computable interface

Finite encodings are the most conceptually direct way to impose combinatorial finiteness, but they are not always the most convenient format for *input* or for certain computations. Miller's framework supplies a complementary algebraic interface: *fringe presentations*. These generalize the idea of presenting a module via generators and relations, where generators are organized as upsets (birth regions) and relations as downsets (death regions).

We begin by recalling the relevant combinatorics of subsets of a poset.

Definition 25 (Upsets and downsets)**.** Let Q be a poset.

(i) An *upset* (or upper set) in Q is a subset $U \subseteq Q$ such that $q \in U$ and $q \leq q'$ imply $q' \in U$.

(ii) A *downset* (or lower set) in Q is a subset $D \subseteq Q$ such that $q \in D$ and $q' \leq q$ imply $q' \in D$.

For $q \in Q$, the *principal upset* generated by q is $\uparrow q := \{q' \in Q : q \leq q'\}$, and the *principal downset* is $\downarrow q := \{q' \in Q : q' \leq q\}$.

Definition 26 (Indicator modules for upsets and downsets)**.** Let Q be a poset and let $S \subseteq Q$. The *indicator module* (or *indicator Q-module*) $\Bbbk[S]$ is the Q-module defined on objects by

$$\Bbbk[S](q) := \begin{cases} \Bbbk & \text{if } q \in S, \\ 0 & \text{if } q \notin S, \end{cases}$$

and on morphisms by the rule that for $q \leq q'$ the structure map $\Bbbk[S](q \leq q')\colon \Bbbk[S](q) \to \Bbbk[S](q')$ is the identity $\Bbbk \to \Bbbk$ when both $q, q' \in S$, and is 0 otherwise. When $S = U$ is an upset, $\Bbbk[U]$ is called an *upset module*; when $S = D$ is a downset, $\Bbbk[D]$ is called a *downset module*.

Remark (Projectives and injectives on a finite poset). When P is a finite poset, the category $\mathbf{Vect}_{\Bbbk}^{P}$ can be identified with the category of finite-dimensional representations of the Hasse quiver of P with commutativity relations (equivalently, with modules over the incidence algebra of P). In this setting, principal upset modules $\Bbbk[\uparrow p]$ are the indecomposable projective objects and principal downset modules $\Bbbk[\downarrow p]$ are the indecomposable injective objects. This is one reason indicator modules are the correct building blocks for computable homological algebra; see [Mil20] for a detailed development in the general poset setting.

The next definition packages a tame module using finitely many upsets and downsets together with a matrix over $\Bbbk$. This is the abstraction of "birth" and "death" data familiar from barcodes, but adapted to general posets.

Definition 27 (Fringe presentation [Mil20, Def. 3.16])**.** Let Q be a poset and let M be a Q-module. A *fringe presentation* of M consists of

(i) a direct sum $F = \bigoplus_{i \in I} \Bbbk[U_i]$ of upset modules,

(ii) a direct sum $E = \bigoplus_{j \in J} \Bbbk[D_j]$ of downset modules, and

(iii) a Q-module homomorphism $\varphi\colon F \to E$

such that $\mathrm{im}(\varphi) \cong M$ and the component maps $\Bbbk[U_i] \to \Bbbk[D_j]$ satisfy a mild connectivity condition (see [Mil20, Def. 3.14]). The fringe presentation is *finite* if the index sets I and J are finite.

Remark (Relation to classical presentations). If $Q = \mathbb{Z}^n$ with the usual partial order, then Q-modules can be interpreted as $\mathbb{Z}^n$-graded modules over the polynomial ring $\Bbbk[x_1, \ldots, x_n]$ (Section 2.4). In that language, upset modules behave like "free/flat" pieces and downset modules behave like "injective" pieces; the map φ organizes generators (birth upsets) and relations (death downsets) in a way that is robust under changes of geometry and does not depend on choosing a coordinate-aligned region decomposition.

Fringe presentations become especially concrete once one writes the map φ in coordinates.

Definition 28 (Monomial matrix [Mil20, Def. 3.17])**.** Let $\varphi\colon F \to E$ be a *finite* fringe presentation with $F = \bigoplus_{i=1}^{m} \Bbbk[U_i]$ and $E = \bigoplus_{j=1}^{r} \Bbbk[D_j]$. Choosing the natural bases on the 1-dimensional stalks of each indicator summand, the map φ can be encoded by an $r \times m$ matrix $\Phi = (\Phi_{ji})$ with entries in $\Bbbk$, whose columns are indexed by the birth upsets U_i and whose rows are indexed by the death downsets D_j. We refer to Φ as a *monomial matrix* for φ.

Lemma 29 (Support condition for monomial matrices [Mil20, Prop. 3.18])**.** *With notation as in Definition 28, one necessarily has $\Phi_{ji} = 0$ whenever $U_i \cap D_j = \varnothing$. Conversely, any matrix $\Phi \in \Bbbk^{r \times m}$ satisfying $\Phi_{ji} = 0$ when $U_i \cap D_j = \varnothing$ determines a fringe presentation map $\varphi\colon F \to E$.*

Example 30 (A single interval in one parameter)**.** Let $Q = \mathbb{R}$ with its total order and consider the interval module $\Bbbk[a, b)$. This module can be realized as the image of a map from the upset module $\Bbbk[a, \infty)$ to the downset module $\Bbbk(-\infty, b)$, and the corresponding monomial matrix is the 1×1 matrix $[1]$. For a direct sum of intervals (a barcode), the matching of births to deaths is reflected by a monomial matrix that is a permutation matrix after choosing orderings of births and deaths; compare [Mil20, Ex. 3.19] and the interval decomposition theorem [CB15].

3.6 What tameness buys you: mathematical consequences

The principal reason tameness is a useful organizing principle is that it is not a single isolated definition but rather part of a web of equivalent finiteness conditions, each suited to different tasks (geometric modeling, algebraic presentation, homological computation). The main structural result is Miller's syzygy theorem.

Theorem 31 (Syzygy theorem [Mil20, Thm. 6.12])**.** *Let Q be a poset and let M be a Q-module. Then M is tame if and only if it admits (and hence admits all of) the following finite descriptions:*

(i) a finite poset encoding subordinate to M;

(ii) a finite fringe presentation of M;

(iii) finite resolutions by upset modules and by downset modules.

Moreover, the constructions can be chosen compatibly with suitable auxiliary geometric structures (e.g. semialgebraic or piecewise-linear settings) when Q sits inside a partially ordered real vector space, and tame morphisms lift to morphisms of the corresponding finite presentations/resolutions.

Remark (Why equivalences matter for software). Theorem 31 has two practical interpretations:

(i) It legitimizes switching among data representations depending on what is being computed. One might build a module from geometric data (suggesting a region-based encoding), transform it to a fringe presentation to expose "birth/death" structure, and then compute derived invariants via an indicator resolution.

(ii) It explains why it is sensible to base a computational pipeline on finite encodings. If a module is tame, then *some* finite encoding exists, and any other finite representation can (in principle) be converted to an encoding suitable for computation.

TAMER-OP is designed around these equivalences: the library implements multiple entry points (fringe-style and region-style) and reduces computations to finite poset models.

A more elementary but extremely important consequence is that, once an encoding is fixed, many constructions can be carried out on the finite poset P and then transported back to Q.

Lemma 32 (Reduction along an encoding)**.** *Suppose $M \cong \pi^* H$ where $\pi\colon Q \to P$ is a finite encoding and H is a P-module. Then every structure map of M is obtained by applying H to the image under π: for $q \le q'$ in Q,*

$$M(q \le q') \;=\; H\big(\pi(q) \le \pi(q')\big).$$

In particular, any invariant of M that is defined functorially from the diagram of vector spaces and structure maps (for example, ranks of structure maps, limits/colimits, or derived functors computed from resolutions) can be computed from H on the finite poset P.

Proof. The identity $M = H \circ \pi$ is the definition of pullback. The final statement follows because π^* is exact (Lemma 17) and because the data of M is completely determined by the data of H together with π. □

3.7 What tameness buys you: algorithmic consequences

The abstract consequences above translate directly into a computational strategy.

Finite linear algebra. If P is finite and H is a P-module with finite-dimensional stalks, then H is specified by:

(i) a finite list of vector spaces $H(p)$, and

(ii) linear maps $H(p \le p')$ subject only to functoriality constraints.

In a finite poset, functoriality can be enforced by recording maps only along cover relations (the Hasse diagram) and composing along paths. This is precisely the kind of finite linear-algebraic data structure on which one can implement kernels, images, cokernels, ranks, and derived constructions.

Homological algebra in a finite abelian category. The abelian category $\mathbf{Vect}_{\Bbbk}^{P}$ has enough projectives and injectives, and its basic building blocks are the principal upset and downset modules (Remark 3.5). Consequently, one can compute projective and injective resolutions by explicit finite algorithms, and then compute Tor and Ext groups by linear algebra on chain complexes. When a Q-module is tame, Theorem 31 guarantees that such finite resolutions exist *upstairs* in Q as well (in the appropriate tame category), but the key computational reduction is that it suffices to resolve the encoded object on a finite poset model.

3.8 Bridge to implementation

The remainder of this thesis is concerned with turning the preceding theory into a usable computational toolkit. The next sections explain:

(i) which concrete finite presentations and encodings TAMER-OP supports,

(ii) how finite encoding posets are constructed from geometric or algebraic input,

(iii) how invariants and derived functors are computed on finite poset models, and

(iv) how these computations feed into a statistics pipeline for data analysis.

In short: tameness provides the *mathematical contract* that makes the entire data-ingestion-to-statistics pipeline finite and computable.

4 Overview of TAMER-OP

4.1 Design goals stated mathematically

4.2 The end-to-end pipeline

4.3 Core mathematical objects in the API

4.4 Architectural layering

4.5 Reproducibility and correctness

5 The presentation formats

The aim of our work is to smoothen a computational tension in the mathematics of tame persistence modules. On the one hand, the natural indexing posets that arise in applications are typically infinite (e.g. $\mathbb{Z}^n$ or $\mathbb{R}^n$), and a persistence module is, definitionally, an infinite functor diagram. On the other hand, any concrete computation must begin from *finite data*. Miller's syzygy theorem (Theorem 31) formalizes the central promise of tameness: a tame module admits multiple *equivalent* finite descriptions (finite encodings, finite fringe presentations, finite indicator resolutions), and one is free to choose whichever description is most computationally convenient for a given task [Mil20].

TAMER-OP turns that promise into a set of concrete data structures and algorithms. This section describes the finite presentation formats implemented in the codebase and explains how they relate to the underlying theory. It is useful to separate three conceptual layers:

(i) **Front-end (Input formats).** These are finite objects that arise naturally from prior constructions and that encode a tame Q-module for an infinite or large poset Q.

(ii) **Encodings (compression to a finite poset).** From a front-end one constructs an order-preserving encoding map $\pi\colon Q \to P$ to a finite poset P, and pushes the module down to a P-module.

(iii) **Finite poset modules (computational core).** Once the module lives on a finite poset P, all downstream computations reduce to finite linear algebra (kernels, images, Ext/Tor, rank queries, Möbius inversion, slicing, and so on). This finite core object is the common target of the conversions in this section and the common input to the algorithms in later sections.

The present section concerns (i), with an emphasis on how each front-end is designed to feed (ii) without forcing an early discretization (such as committing to a large bounding box in $\mathbb{Z}^n$ or an overly fine grid in $\mathbb{R}^n$). Although we mention the conversion routines that produce encodings, the encoding algorithms themselves are deferred to Section 6.

5.1 Philosophy: multiple equivalent front-ends

A practical library for multiparameter persistence needs to accept input from several sources. For example:

- A user might already have a *finite* poset model (an encoding poset built elsewhere) and wants to compute invariants on it.
- A discrete pipeline (e.g. a $\mathbb{Z}^n$ multifiltration) naturally produces multigraded data resembling commutative algebra, where presentations and resolutions are most naturally expressed using specialized building blocks on $\mathbb{Z}^n$.
- A continuous pipeline (e.g. a PL or semialgebraic construction over $\mathbb{R}^n$) often yields piecewise-constant behavior on polyhedral regions, and it is unnatural (and numerically risky) to snap this immediately to a grid.

The unifying observation, following [Mil20], is that these sources can all be expressed in a common pattern: *indicator modules for upward-closed and downward-closed sets, glued by a matrix.* This motivates the following heuristic definition.

Definition 33 (Presentation object in TAMER-OP)**.** Fix a poset Q. A *presentation object* for a Q-module is a finite package of data that determines a tame Q-module functorially, in the sense that:

(i) for each parameter value $q \in Q$, one can compute, even if implicitly, the fiber $M(q)$, and for comparable $q \leq q'$ one can compute the structure map $M(q \leq q')$;

(ii) the module can be encoded by a finite constant subdivision obtained from the same data;

(iii) the representation is stable under the basic operations used in the library (direct sums, restriction to chains, and the finite encoding step).

This definition is intentionally not a purely mathematical one; it is meant to capture what the code must accomplish. In practice, a presentation object is something like a `struct` whose fields are (i) a finite list of "birth" shapes (upsets), (ii) a finite list of "death" shapes (downsets), and (iii) a scalar matrix specifying how births map into deaths. The precise meaning of "shape" depends on the ambient poset:

- for a finite poset P, shapes are subsets of the vertex set, represented efficiently as bitmasks;
- for $\mathbb{Z}^n$, shapes are the indecomposable *flats* and *injectives* from Miller's $\mathbb{Z}^n$ theory [Mil20];
- for $\mathbb{R}^n$ in PL settings, shapes are unions of polyhedral upsets/downsets, represented by rational half-space data (or by specialized axis-aligned primitives).

A key design decision, again following Miller's principles, is that TAMER-OP does *not* attempt to compute invariants directly on the infinite posets $\mathbb{Z}^n$ or $\mathbb{R}^n$ from these presentations. Instead, the role of the presentation is to provide enough structure to build a finite encoding poset P and an encoding map $\pi\colon Q \to P$. Once P is available, the library works with the explicit P-modules and standard finite algorithms in the abelian category $\mathbf{Vect}_{\Bbbk}^{P}$ (Section **??**).

5.2 Finite fringe presentations

The most literal computational realization of Miller's framework is obtained when the indexing poset is already finite. In that case, a P-module is simply a finite diagram of vector spaces and linear maps, and indicator modules for upsets/downsets become finitely supported objects. This case plays two roles in TAMER-OP:

(i) it provides a clean, fully explicit core format for correctness testing and for downstream algebra; and

(ii) it is the *output* format of the encoding step, even when the original module lived on $\mathbb{Z}^n$ or $\mathbb{R}^n$.

Finite posets and bitmask upsets/downsets

In the codebase (see `FiniteFringe.jl`), a finite poset P is represented by a finite set of vertices $\{1, \ldots, N\}$ together with a cached order relation. Upsets and downsets are represented by bit-vectors of length N.

Definition 34 (Finite-poset upsets/downsets as data)**.** Let P be a finite poset with vertex set $\{1, \ldots, N\}$. A *finite upset datum* is a bitmask $U \in \{0,1\}^N$ such that $U(p) = 1$ and $p \leq q$ imply $U(q) = 1$. A *finite downset datum* is a bitmask $D \in \{0,1\}^N$ such that $D(q) = 1$ and $p \leq q$ imply $D(p) = 1$.

In addition to explicit bitmasks, the implementation supports closures of arbitrary subsets: given a list of generators, one can compute the smallest upset (resp. downset) containing them by repeatedly closing upward (resp. downward). This is useful when constructing projective/injective covers later, because principal upsets $\Bbbk[\uparrow p]$ and principal downsets $\Bbbk[\downarrow p]$ appear repeatedly (cf. Remark 3.5).

Fringe data and degreewise evaluation

The main presentation format on a finite poset is a finite fringe presentation.

Definition 35 (Finite fringe presentation data)**.** Let P be a finite poset and fix a field $\Bbbk$. A *finite fringe presentation datum* consists of:

(i) a finite list of upsets $U_1, \dots, U_m \subseteq P$ (birth regions),

(ii) a finite list of downsets $D_1, \dots, D_r \subseteq P$ (death regions),

(iii) a matrix $\Phi \in \Bbbk^{r \times m}$ with the *support condition* $\Phi_{ji} = 0$ whenever $U_i \cap D_j = \varnothing$,

and it defines a P-module as the image of the induced map

$$\varphi\colon \bigoplus_{i=1}^{m} \Bbbk[U_i] \longrightarrow \bigoplus_{j=1}^{r} \Bbbk[D_j]$$

whose component maps $\Bbbk[U_i] \to \Bbbk[D_j]$ are multiplication by Φ_{ji} on $U_i \cap D_j$ and zero elsewhere (compare [Mil20, Defs. 3.16–3.17]).

Degreewise evaluation is particularly direct. For a vertex $p \in P$, define the active column set $I(p) = \{i : p \in U_i\}$ and active row set $J(p) = \{j : p \in D_j\}$. Evaluating φ at p yields a linear map

$$\varphi_p\colon \Bbbk^{I(p)} \longrightarrow \Bbbk^{J(p)}$$

whose matrix is the submatrix $\Phi_{J(p),I(p)}$. The fiber of the module at p is

$$M(p) = \operatorname{im}(\varphi_p)$$

so

$$\dim_{\Bbbk} M(p) = \operatorname{rank}\big(\Phi_{J(p),I(p)}\big).$$

In TAMER-OP, this is implemented directly: membership in each U_i and D_j is a bit-test, so constructing $I(p)$ and $J(p)$ is efficient, and rank computations are performed over exact coefficients (typically $\mathbb{Q}$) to avoid numerical rank instability.

Remark (Why the support condition matters computationally). The support condition $\Phi_{ji} = 0$ when $U_i \cap D_j = \varnothing$ is the finite-poset analog of a very practical constraint: it guarantees that evaluation at each p never introduces spurious maps between zero and nonzero stalks. In the code, constructors enforce this condition and can optionally store Φ as a sparse matrix when the support is sparse. This matters because, even when m and r are moderate, the dense matrix Φ can be the dominant memory cost.

From fringe presentations to explicit P-modules

A fringe presentation is already a finite representation, but many downstream algorithms prefer an explicit representation of a P-module in terms of:

(i) dimensions $\dim M(p)$ at each vertex p, and

(ii) linear maps along cover relations in the Hasse diagram, with all other structure maps obtained by composition.

This explicit format is implemented in the codebase as the `PModule` type (defined in `Modules.jl`). The conversion `pmodule_from_fringe` (in `IndicatorResolutions.jl`) follows the mathematics suggested by Proposition 2.1: choose bases for each fiber $M(p)$ and then define the map $M(p \le q)$ as the unique linear map making the relevant inclusion squares commute.

Algorithmically, the conversion has two conceptual steps:

(i) For each vertex p, compute a basis for the image $\mathrm{im}(\varphi_p) \subseteq \Bbbk^{J(p)}$. In exact arithmetic this can be done by row-reduction of $\Phi_{J(p),I(p)}$ and extracting a column-space basis.

(ii) For each cover relation $p \prec q$ in P, compute the induced linear map $M(p) \to M(q)$ by expressing the image basis vectors at p in the basis at q via the natural inclusions $\Bbbk^{J(p)} \hookrightarrow \Bbbk^{J(q)}$ coming from downset functoriality.

To keep this step practical, the implementation caches intermediate basis computations and reuses them when multiple edges share endpoints. This conversion is also the place where exact arithmetic over $\mathbb{Q}$ is most valuable: small numerical errors in rank computations would otherwise lead to inconsistent dimensions and broken functoriality.

5.3 $\mathbb{Z}^n$ presentations

When the indexing poset is $Q = \mathbb{Z}^n$ with coordinatewise order, persistence modules become closely related to multigraded commutative algebra (Section 2.4). Concretely, $\mathbb{N}^n$-indexed modules are equivalent to multigraded modules over $\Bbbk[x_1, \ldots, x_n]$ (Proposition 2.4), and even for $\mathbb{Z}^n$-indexing, many of the same structural ideas persist [CZ09, Oud15].

A naive approach to computation on $\mathbb{Z}^n$ would restrict to a large finite box and treat it as a finite poset. This is workable for toy problems but scales poorly, since the number of lattice points grows exponentially in n. TAMER-OP therefore uses a presentation format tailored to Miller's $\mathbb{Z}^n$ theory: *flange presentations* built from indecomposable *flats* and *injectives* [Mil20, §5]. The guiding idea is the same as in finite fringe presentations, but the building blocks are chosen so that membership and intersection are simple and so that finite encodings can be constructed without enumerating a huge grid.

Faces, flats, and injectives

Fix $n \geq 1$ and write $\mathbf{e}_1, \ldots, \mathbf{e}_n$ for the standard basis of $\mathbb{Z}^n$. A *face* (in the sense used in the code and in [Mil20]) is a choice of a subset of coordinates along which we allow bi-infinite translation.

Definition 36 (Face subgroup)**.** Let $\tau \subseteq \{1, \ldots, n\}$. Define the subgroup

$$\mathbb{Z}^\tau := \{v \in \mathbb{Z}^n : v_i = 0 \text{ for } i \notin \tau\}.$$

Equivalently, $\mathbb{Z}^\tau$ is spanned by $\{\mathbf{e}_i : i \in \tau\}$.

Definition 37 (Indecomposable flat and injective shapes)**.** Fix $\tau \subseteq \{1, \ldots, n\}$ and a basepoint $\mathbf{b} \in \mathbb{Z}^n$. Define subsets of $\mathbb{Z}^n$:

$$F(\mathbf{b}, \tau) := \mathbf{b} + \mathbb{N}^n + \mathbb{Z}^\tau, \qquad E(\mathbf{b}, \tau) := \mathbf{b} + \mathbb{Z}^\tau - \mathbb{N}^n.$$

Then $F(\mathbf{b}, \tau)$ is an upset in $\mathbb{Z}^n$, and $E(\mathbf{b}, \tau)$ is a downset. The corresponding indicator modules $\Bbbk[F(\mathbf{b}, \tau)]$ and $\Bbbk[E(\mathbf{b}, \tau)]$ are the basic indecomposable "flat" and "injective" building blocks used in flange presentations [Mil20].

In the Julia implementation (`FlangeZn.jl`), a face τ is stored as a bitmask, and a flat/injective is stored as the pair $(\mathbf{b}, \tau)$ together with an optional identifier. Membership tests reduce to coordinatewise inequalities after projecting away the τ directions. This makes evaluation cheap even when n is moderate.

Flange presentations

Definition 38 (Flange presentation over $\mathbb{Z}^n$)**.** A $\mathbb{Z}^n$ *flange presentation* (over a field $\Bbbk$) consists of:

(i) a finite list of flat indicators $\Bbbk[F_1], \ldots, \Bbbk[F_m]$ with $F_i = F(\mathbf{b}_i, \tau_i)$,

(ii) a finite list of injective indicators $\Bbbk[E_1], \ldots, \Bbbk[E_r]$ with $E_j = E(\mathbf{c}_j, \sigma_j)$,

(iii) a scalar matrix $\Phi \in \Bbbk^{r \times m}$ (rows indexed by injectives, columns by flats),

which determines a $\mathbb{Z}^n$-module

$$M := \mathrm{im}\left(\varphi\colon \bigoplus_{i=1}^{m} \Bbbk[F_i] \longrightarrow \bigoplus_{j=1}^{r} \Bbbk[E_j]\right),$$

where the component map $\Bbbk[F_i] \to \Bbbk[E_j]$ is multiplication by Φ_{ji} on $F_i \cap E_j$ and zero otherwise (compare [Mil20, Rems. 5.5 and 5.11, Def. 5.14]).

The degreewise evaluation mirrors the finite-poset case quite closely. For $\mathbf{g} \in \mathbb{Z}^n$, let $I(\mathbf{g}) = \{i : \mathbf{g} \in F_i\}$ and $J(\mathbf{g}) = \{j : \mathbf{g} \in E_j\}$. Then evaluation at $\mathbf{g}$ yields a linear map

$$\varphi_{\mathbf{g}} \colon \Bbbk^{I(\mathbf{g})} \longrightarrow \Bbbk^{J(\mathbf{g})}$$

with matrix $\Phi_{J(\mathbf{g}),I(\mathbf{g})}$, and hence

$$\dim_{\Bbbk} M(\mathbf{g}) = \operatorname{rank}\big(\Phi_{J(\mathbf{g}),I(\mathbf{g})}\big).$$

In the code, this is implemented by extracting the active row and column indices via membership tests and then computing the rank of the resulting submatrix. For correctness, the default coefficient field used throughout the current implementation is $\mathbb{Q}$, with rank computed by exact row reduction.

Remark (Automatic enforcement of the monomial support condition). As with finite fringe presentations, flange presentations should respect a support condition: entries corresponding to empty intersections $F_i \cap E_j = \varnothing$ do not contribute to any degree and are safely forced to zero. The `Flange` constructor in `FlangeZn.jl` enforces this by explicitly zeroing such entries, which prevents subtle bugs where a user accidentally supplies a dense matrix with spurious coefficients. This is also a mild but useful performance improvement: it increases sparsity in common cases where the intersection pattern is already sparse.

Why flanges are the preferred $\mathbb{Z}^n$ front-end

Flange presentations are not the only finite representation of tame $\mathbb{Z}^n$-modules, but they have several advantages that justify their role as the primary $\mathbb{Z}^n$ front-end:

- **They are theory-native.** They are the $\mathbb{Z}^n$ analog of fringe presentations and arise naturally from Miller's development of finitely determined modules [Mil20].
- **They avoid bounding boxes.** One can often build a finite encoding poset using only the finite list of flats and injectives, rather than enumerating a large grid of degrees. This design goal strongly influences the encoding algorithms in Section 6.
- **They are compatible with commutative algebra tooling.** In multigraded algebra, flat and injective objects play roles analogous to localizations and injective hulls; they align well with resolution-based invariants.

The cost is that a flange presentation is less immediately geometric than a polyhedral $\mathbb{R}^n$ presentation. TAMER-OP therefore treats flanges as a specialized, highly efficient format for lattice-indexed modules, not as a universal one.

5.4 $\mathbb{R}^n$ and piecewise-linear presentations

Many persistence modules arising from real-parameter filtrations are *piecewise constant* on regions cut out by a finite set of geometric constraints. In the simplest PL settings, these constraints come from finitely many affine half-spaces and their intersections. This observation is reflected both in the theory of constructible persistence modules and in sheaf-theoretic approaches [KS18, Mil23]. In TAMER-OP, the corresponding front-end is a *PL fringe presentation*: a fringe presentation whose upsets and downsets are represented as polyhedral regions in $\mathbb{R}^n$ (or, in a specialized fast path, as unions of axis-aligned boxes).

Geometric upsets and downsets

Let $Q = \mathbb{R}^n$ with the coordinatewise order. A subset $U \subseteq \mathbb{R}^n$ defines an upset indicator module $\Bbbk[U]$ only when U is upward closed; likewise D must be downward closed. For computation, we need concrete representations of such sets and efficient membership tests.

The codebase supports two PL backends (see `PLPolyhedra.jl` and `PLBackend.jl`), reflecting a tradeoff:

- **General polyhedral backend.** Upsets and downsets are represented as finite unions of rational convex polyhedra (typically specified by half-space inequalities). This is flexible and matches the theoretical PL picture but can require heavier polyhedral geometry routines.

- **Axis-aligned box backend.** Upsets and downsets are represented as unions of axis-aligned boxes (rectangles in $n = 2$). This case supports very fast region decompositions and is well adapted to many practical filtrations where constraints are coordinate thresholds.

In both backends, the intent is that the sets are *certifiably* upward or downward closed. For the purposes of this thesis, it suffices to emphasize that each shape supports:

(i) membership testing $q \in U_i$ or $q \in D_j$,

(ii) intersection testing (needed to enforce the matrix support condition),

(iii) extraction of a finite set of "boundary features" that will generate the constant subdivision used for encoding.

The encoding algorithms that exploit these features are postponed to Section 6.

PL fringe presentations

With geometric upsets and downsets in hand, the PL fringe presentation is formally identical to the finite fringe presentation, except that $Q = \mathbb{R}^n$ is infinite and the shapes are geometric rather than combinatorial.

Definition 39 (PL fringe presentation)**.** A *PL fringe presentation* over $Q = \mathbb{R}^n$ consists of:

(i) finitely many geometric upsets $U_1, \ldots, U_m \subseteq \mathbb{R}^n$,

(ii) finitely many geometric downsets $D_1, \ldots, D_r \subseteq \mathbb{R}^n$,

(iii) a matrix $\Phi \in \Bbbk^{r \times m}$ satisfying $\Phi_{ji} = 0$ whenever $U_i \cap D_j = \varnothing$,

and it defines the $\mathbb{R}^n$-module

$$M := \operatorname{im}\left(\bigoplus_{i=1}^{m} \Bbbk[U_i] \xrightarrow{\Phi} \bigoplus_{j=1}^{r} \Bbbk[D_j] \right).$$

As before, the fiber dimension at $q \in \mathbb{R}^n$ is computed by restricting Φ to active rows and columns and taking a rank. In practice, however, such pointwise queries are only a diagnostic tool. The primary use of a PL fringe presentation is as input to a finite encoding step: the shapes U_i and D_j determine a finite subdivision of $\mathbb{R}^n$ into regions on which all membership patterns are constant, and hence on which M is constant. This recovers the "finite regions of behavior" intuition behind tameness [Mil20].

Remark (Exactness and geometric robustness)*.* The PL setting is where numerical issues most easily corrupt a computation, because region boundaries are defined by equalities and inequalities. The current implementation therefore emphasizes rational geometry where possible: polyhedral constraints are stored with rational coefficients, and rank computations are performed over $\mathbb{Q}$. This does not eliminate all geometric subtleties, as degenerate intersections still require care. However, it does avoid potential failures in persistent computations due to incorrect ranks induced by floating-point instability near boundary cases.

5.5 Conversions between formats

The core promise of this section is that the front-ends described above are not siloed. They are designed so that, after a finite encoding is constructed, they can all be converted to the same finite core format. The overarching conversion pattern is:

(front-end presentation on Q)
$\Downarrow$ (construct finite encoding $\pi\colon Q \to P$)
(fringe presentation on finite P)
$\Downarrow$ (basis extraction + cover-edge maps)
(explicit P-module `PModule`)

The theoretical justification is that the encoding poset captures the module up to isomorphism in the tame category (Definition 21 and Theorem 22), and all further invariants computed by TAMER-OP factor through this finite model (Lemma 32).

Finite-poset fringe → explicit P-module

This is the simplest conversion and was already outlined in Subsection 5.2. It is implemented by `pmodule_from_fringe`, which computes image bases at each vertex and induced maps on cover relations.

$\mathbb{Z}^n$ flange → finite encoding → P-module

For $\mathbb{Z}^n$ flanges, the key step is constructing an encoding without enumerating a large portion of the lattice. At a conceptual level, the conversion uses the finite family of sets

$$\{F_i\}_{i=1}^{m} \;\cup\; \{\mathbb{Z}^n \setminus E_j\}_{j=1}^{r}$$

to define a *signature* map

$$\mathrm{sig}\colon \mathbb{Z}^n \longrightarrow \{0,1\}^{m+r}, \qquad \mathrm{sig}(\mathbf{g}) = \big(\mathbf{1}_{\mathbf{g}\in F_1}, \dots, \mathbf{1}_{\mathbf{g}\in F_m}, \mathbf{1}_{\mathbf{g}\notin E_1}, \dots, \mathbf{1}_{\mathbf{g}\notin E_r}\big).$$

Points with the same signature lie in the same constant region for the presentation, hence the same fiber of an encoding map. The nontrivial work is to show that only finitely many signatures occur under suitable finiteness hypotheses and to construct a finite poset P of regions ordered by reachability. This is precisely the kind of finite constant subdivision that Miller uses to define tame morphisms and encodings [Mil20]. The algorithmic realization in the codebase is implemented in `ZnEncoding.jl`.

Once P and π are constructed, the flange is pushed down to a fringe presentation on P by evaluating each flat and injective region on a representative for each region of P, producing explicit upsets and downsets in the finite poset. The remainder of the pipeline (fringe → `PModule`) is then identical to the finite case.

PL fringe → finite encoding → P-module

The PL case follows the same signature philosophy, but now using geometric objects. The constant subdivision is obtained from the finite set of boundaries that define the polyhedral upsets and downsets, yielding finitely many regions when the input is PL or semialgebraic. The polyhedral backend (`PLPolyhedra.jl`) and the axis-aligned backend (`PLBackend.jl`) implement different concrete strategies for producing the same output: a finite poset P, an encoding map $\pi\colon \mathbb{R}^n \to P$ (with a point-location routine), and a pushed-down fringe presentation on P.

Remark (Why conversions are staged). It is tempting to attempt a direct conversion from an infinite presentation (flange or PL fringe) to an explicit P-module by sampling many points and inferring structure maps. We avoid this on TAMER-OP. The staged approach (front-end → encoding → fringe on P → explicit P-module) mirrors the theory and isolates sources of error. Here, any mismatch can be diagnosed at the level of signatures and regions before committing to linear algebra on the finite poset.

5.6 Validation and other sanity checks

Compact presentations are powerful precisely because they hide large implicit structure. But, that also makes them fragile. A small inconsistency in the input can produce a module that is not functorial, not tame, or simply not the module the user intended. TAMER-OP therefore incorporates validation at multiple layers. The checks fall into two categories: *structural* checks that are inexpensive and always enabled in constructors, and *diagnostic* checks that can be toggled because they may be expensive.

Structural checks

- **Poset correctness.** For a finite poset, we check that the stored relation is reflexive and transitive and that it is antisymmetric up to equality of vertices. Cover relations are derived from the order and are used as the canonical generating set of morphisms.
- **Upset/downset closure.** For finite-poset upsets and downsets represented as bitmasks, constructors and closure routines ensure upward-closed and downward-closed properties by construction.

- **Monomial (support) conditions.** For each presentation format, nonzero matrix entries are forbidden when the corresponding birth and death shapes do not intersect. Constructors enforce this condition automatically: in the finite fringe format, a check verifies $\Phi_{ji} = 0$ when $U_i \cap D_j = \varnothing$; in the $\mathbb{Z}^n$ flange format, the constructor explicitly zeros such entries; and in the PL formats, intersection tests serve the same role. These checks directly implement the support constraints that make a matrix correspond to an actual natural transformation of indicator modules [Mil20, Prop. 3.18].

Diagnostic checks (optional)

- **Spot-check functoriality.** For presentations that support evaluation, one can sample comparable pairs $q \leq q'$ and verify that the induced maps satisfy basic functorial constraints along short chains.
- **Encoding correctness tests.** After building an encoding $\pi\colon Q \to P$, we can sample points $q, q' \in Q$ with $\pi(q) = \pi(q')$ and verify that the presentation evaluates to the same fiber data at q and q'. This corresponds directly to the mathematical requirement that M be constant on fibers of π (Remark 3.2).
- **Rank certification.** When working over $\mathbb{Q}$, ranks are computed by exact elimination; this can be used to certify that computed fiber dimensions and structure maps are consistent. In later sections, the same philosophy is used to certify minimality of resolutions on finite posets.

Bridge to the encoding algorithms

At this point we have described the finite data structures that represent tame modules before encoding. The next section addresses the algorithmic core of TAMER-OP: given one of these presentation formats, how does one construct a finite encoding poset P and an encoding map $\pi\colon Q \to P$ efficiently and robustly, and how does one push the module down to P so that all subsequent invariants can be computed by finite linear algebra?

6 Finite encoding constructors

A recurring theme in Chapters 2–3 is that "tameness" is not merely a finiteness hypothesis but a promise that persistence modules admit *finite combinatorial models*. In Miller's language, a tame Q-module M admits a *finite poset encoding* $\pi\colon Q \to P$ with P finite and $M \cong \pi^* H$ for some P-module H (Definition 15), and this is equivalent to the existence of finite fringe-style descriptions (Theorem 31) [Mil20]. The purpose of this section is to explain how TAMER-OP *constructs* such a finite encoding from the finite presentations introduced in the previous section.

A fringe or flange presentation is a finite collection of geometric regions in parameter space (upsets and downsets, often polyhedral) together with a matrix of coefficients. The encoding problem asks for a finite poset P of "regions of constant behavior" together with a map $\pi\colon Q \to P$ that collapses each constant region to a single element. Once this is achieved, the computational task shifts decisively: all subsequent operations on M (kernels, images, ranks, resolutions, derived functors, and so on) reduce to finite linear algebra on the encoded P-module, as in Lemma 32.

Algorithmically, TAMER-OP implements encodings in three ambient regimes:

- **Lattice encodings** for $Q = \mathbb{Z}^n$ (implemented in `ZnEncoding.jl`), where presentations are built from indicator flats and injectives (Section 5.3).
- **Axis-aligned piecewise-linear encodings** for $Q = \mathbb{R}^n$ with coordinatewise order and regions described by finite unions of orthants (implemented in `PLBackend.jl`).
- **General polyhedral PL encodings** for $Q = \mathbb{R}^n$ where regions are unions of rational polyhedra (implemented in `PLPolyhedra.jl`), used when axis-aligned structure is insufficient.

All three share a single organizing principle: *compute a finite partition of parameter space on which the relevant indicator data are constant, then collapse cells with the same signature.*

6.1 The encoding problem

Fix an ambient poset Q (typically $Q = \mathbb{Z}^n$ or $Q = \mathbb{R}^n$ with coordinatewise order). The immediate input to an encoding constructor in TAMER-OP is a finite presentation in one of the formats from Section 5. For the present section it is convenient to phrase this input uniformly as follows.

Input data model. A finite fringe-style presentation provides:

(i) a finite collection of *birth upsets* $U_1, \dots, U_m \subseteq Q$,

(ii) a finite collection of *death downsets* $D_1, \dots, D_r \subseteq Q$,

(iii) a matrix $\Phi \in \Bbbk^{r \times m}$ specifying a map $\varphi\colon \bigoplus_{i=1}^m \Bbbk[U_i] \to \bigoplus_{j=1}^r \Bbbk[D_j]$ (Lemmas 29 and Definition 28).

In the lattice implementation these U_i and D_j are not literally stored as arbitrary subsets; instead they come from structured families (indicator flats and injectives) that admit fast membership tests and finite boundary descriptions (`IndFlat` and `IndInj` in `FlangeZn.jl`). In the PL implementations the upsets and downsets are represented as finite unions of structured polyhedral sets (orthants in `PLBackend.jl`, general rational polyhedra in `PLPolyhedra.jl`).

Desired output. An encoding constructor produces:

(i) a finite poset P (the *encoding poset*),

(ii) a poset morphism $\pi\colon Q \to P$ (the *encoding map*),

(iii) a P-module $H\colon P \to \mathbf{Vect}_\Bbbk$, such that the presented Q-module factors as $M \cong \pi^* H$.

In code, the output is typically packaged as `(P, H, pi)` or `(P, Hs, pi)` when encoding several modules at once. Here H is stored as a finite poset module in `FiniteFringe.jl` (usually a `FringeModule` on P), and π is stored as a lightweight object supporting point location queries (for example `ZnEncodingMap`).

A methodological choice. There is no unique "best" encoding in general. Even when a tame encoding exists, it need not be canonical, and the problem of finding a minimal encoding can be computationally expensive or ill-posed under different equivalence notions. TAMER-OP therefore adopts a *presentation-relative* encoding strategy: it constructs an encoding that is guaranteed to be subordinate to the *specific* finite family of upsets and downsets appearing in the chosen input format. This choice matches the computational contract of Miller's equivalences: theorem-level existence is supplied by tameness [Mil20], while the encoding constructor is a deterministic algorithm whose complexity can be analyzed in terms of the size of the given finite presentation.

6.2 Region/signature idea

The core encoding mechanism used throughout TAMER-OP can be stated abstractly without reference to a particular ambient domain.

Definition 40 (Signature map associated to a finite fringe interface)**.** Let Q be a poset and suppose we are given upsets $U_1, \dots, U_m \subseteq Q$ and downsets $D_1, \dots, D_r \subseteq Q$. Define functions

$$\chi_{U_i}(q) := \begin{cases} 1 & q \in U_i, \\ 0 & q \notin U_i, \end{cases} \qquad \chi_{\overline{D_j}}(q) := \begin{cases} 1 & q \notin D_j, \\ 0 & q \in D_j, \end{cases}$$

and define the *signature* of $q \in Q$ to be the bit vector

$$\sigma(q) := \big(\chi_{U_1}(q), \dots, \chi_{U_m}(q)\,;\, \chi_{\overline{D_1}}(q), \dots, \chi_{\overline{D_r}}(q)\big) \in \{0,1\}^{m+r}.$$

The reason for encoding downsets via their complements is monotonicity: if D is a downset, then $Q \setminus D$ is an upset, hence order-preserving in the same direction as birth regions.

Lemma 41 (Monotonicity of signatures)**.** *The signature map $\sigma\colon Q \to \{0,1\}^{m+r}$ in Definition 40 is order-preserving when $\{0,1\}^{m+r}$ is ordered coordinatewise.*

Proof. If $q \le q'$ in Q and $q \in U_i$ for some upset U_i, then $q' \in U_i$ by upward closure, so $\chi_{U_i}(q) \le \chi_{U_i}(q')$. If $q \le q'$ and $q \notin D_j$ for a downset D_j, then $q' \notin D_j$ as well, because membership in a downset can only be lost when moving upward. Equivalently, $Q \setminus D_j$ is an upset. Hence $\chi_{\overline{D_j}}(q) \le \chi_{\overline{D_j}}(q')$ for all j. □

The following encoding construction follows naturally from the lemma.

Proposition (Encoding via realized signatures)**.** *Let $P := \mathrm{im}(\sigma) \subseteq \{0,1\}^{m+r}$ with the induced partial order, and let $\pi := \sigma\colon Q \to P$ (followed by the projection onto the image). Then for each i and j, the indicator modules $\Bbbk[U_i]$ and $\Bbbk[D_j]$ factor through π. Consequently, any fringe presentation map $\varphi\colon \bigoplus_i \Bbbk[U_i] \to \bigoplus_j \Bbbk[D_j]$ induces a P-module presentation whose pullback along π recovers the original one.*

Proof sketch. For each i, define an upset $\widehat{U}_i \subseteq P$ by

$$\widehat{U}_i := \{p \in P : \text{the } i\text{th upset-bit of } p \text{ equals } 1\}.$$

Because the ith coordinate is monotone in P, this is indeed an upset in P, and by construction $\pi^{-1}(\widehat{U}_i) = U_i$. Thus $\Bbbk[U_i] \cong \pi^*\Bbbk[\widehat{U}_i]$. Similarly, define a downset $\widehat{D}_j \subseteq P$ by declaring $p \in \widehat{D}_j$ if the $(m+j)$th bit of p (the complement-of-downset bit) equals 0, so that $\pi^{-1}(\widehat{D}_j) = D_j$. The map φ is specified stalkwise by the matrix Φ on intersections $U_i \cap D_j$, and those intersections are preserved under pullback because each U_i and D_j is a union of fibers of π. □

Proposition 6.2 conceptually encompasses encoding as a whole: *once* the set of realized signatures is finite, one has a finite encoding. The real work is therefore to compute $\mathrm{im}(\sigma)$ efficiently from a structured description of the U_i and D_j, and to equip the realized signatures with an implementation-friendly poset structure and a fast point-location map π.

From a geometric standpoint, this signature construction is a concrete incarnation of the "constant subdivision" idea in Miller's definition of tame morphisms [Mil20]. The regions on which all indicator data are constant form a finite stratification of parameter space, analogous in spirit to the stratifications appearing in constructible sheaf approaches [KS18, Mil23], though TAMER-OP works directly with the combinatorics of indicator regions rather than with microsupport.

6.3 Encoding algorithms by ambient domain

This subsection describes how TAMER-OP computes the realized signatures $\mathrm{im}(\sigma)$ in each supported regime. The three backends share the same high-level structure but differ in how they represent regions and how they enumerate constant cells.

(A) Lattice encoding for $Q = \mathbb{Z}^n$

The lattice backend is designed for presentations arising from $\mathbb{Z}^n$-indexed modules, where indicator regions are built from algebraic data that are intrinsically discrete.

Indicator families and membership tests. In `FlangeZn.jl`, the basic structured regions are:

- `IndFlat` objects, which encode certain coordinatewise upsets by inequalities of the form $g_i \ge b_i$ on a specified subset of coordinates; and
- `IndInj` objects, which encode coordinatewise downsets by inequalities $g_i \le b_i$ on a specified subset of coordinates.

Both admit $O(n)$ membership checks (`in_flat` and `in_inj`), but the key point is that their *boundaries* occur only at finitely many integer coordinate values extracted from the b_i in the presentation.

Critical coordinates and a product cell complex. The encoding constructor `encode_poset_from_flanges` in `ZnEncoding.jl` begins by collecting, for each axis $k = 1, \dots, n$, a sorted list of *critical coordinates* at which some membership predicate changes. Concretely:

- for each flat constraint $g_k \ge b_k$, the threshold b_k is critical;

- for each injective constraint $g_k \le b_k$, the complement threshold $b_k + 1$ is critical, because in $\mathbb{Z}$ the complement of $g_k \le b_k$ is $g_k \ge b_k + 1$.

Call these lists $C_k \subset \mathbb{Z}$. They define a finite product cell structure on $\mathbb{Z}^n$ by decomposing each coordinate axis into slabs between successive critical values. The resulting *grid of cells* has size

$$N_{\text{cell}} = \prod_{k=1}^{n} (|C_k| + 1).$$

On each cell, the truth values of all inequalities defining all flats and injectives are constant, hence the signature σ is constant.

Signature enumeration by incremental scanning. A direct approach would evaluate all $m + r$ membership predicates on a representative lattice point of each cell, costing $O((m + r)\, n\, N_{\text{cell}})$ in the worst case. The implementation instead performs a single scan through the grid, updating the signature incrementally as it crosses critical coordinates. The key mechanism is:

(i) precompute, for each axis and each critical coordinate, which flats or injective complements toggle their constraint status when crossing that coordinate; and

(ii) traverse the grid in a snake-like boustrophedon order so that successive cells differ in only one coordinate direction.

For each cell visited, the scanner updates a small set of counters (how many constraints of each flat are still unsatisfied, and how many constraints of each injective have been violated) and maintains bitvectors encoding the current (y, z) signature. Each visited cell contributes its signature to a hash table of seen signatures, and the scan optionally records a lookup table `cell_to_region` mapping cell indices to signature-region indices.

Output. The scan returns the list of realized signatures and a map $\pi\colon \mathbb{Z}^n \to P$ implemented as a `ZnEncodingMap`. The encoding poset P can be represented either:

- implicitly, as a `SignaturePoset` where $p \le p'$ is tested by subset operations on the stored signature bitvectors; or
- explicitly, as a `FinitePoset` whose order relation is materialized as a BitMatrix for faster downstream closure operations.

This choice is controlled by `poset_kind` in `EncodingOptions`.

(B) Axis-aligned PL encoding for $Q = \mathbb{R}^n$

The axis-aligned PL backend is the continuous analogue of the lattice backend. Here $Q = \mathbb{R}^n$ with the coordinatewise order, and the input regions are finite unions of *orthants* described by threshold vectors.

Region representation. In `PLBackend.jl`, an upset is represented as a finite set of "corners" $\ell^{(1)}, \dots, \ell^{(s)} \in \mathbb{R}^n$, encoding the union

$$U = \bigcup_{t=1}^{s} \{x \in \mathbb{R}^n : x \ge \ell^{(t)}\},$$

and similarly a downset is represented by finitely many "upper corners" $u^{(1)}, \dots, u^{(t)}$ encoding

$$D = \bigcup_{t=1}^{T} \{x \in \mathbb{R}^n : x \le u^{(t)}\}.$$

This matches a geometry that should be common in PL persistence constructions, where births and deaths are controlled by finitely many coordinate thresholds.

Coordinate hyperplane subdivision. The encoding constructor `encode_fringe_boxes` collects, for each axis k, all threshold coordinates appearing in any $\ell_k^{(t)}$ or $u_k^{(t)}$. These values define a subdivision of $\mathbb{R}$ into open intervals, and hence a subdivision of $\mathbb{R}^n$ into open axis-aligned boxes (cells). As in the lattice case, the signature σ is constant on each cell.

Representative points and strictness. A subtlety in the continuous setting is boundary handling: if a representative point lies on a threshold hyperplane, membership in a closed upset/downset can be ambiguous relative to the intended open cell. TAMER-OP resolves this by choosing a point in the *interior* of each cell, implemented via a small offset parameter `strict_eps` in `EncodingOptions`. This makes the signature assignment robust and deterministic.

Scanning and output. The axis-aligned backend performs a scan through the grid of cells and records realized signatures, using essentially the same boustrophedon mechanism as the lattice backend (implemented in `_collect_cell_signatures_scan`). The output is a finite poset P on realized signatures and an encoding map $\pi\colon \mathbb{R}^n \to P$ implemented as a `BoxEncodingMap`.

(C) General polyhedral PL encoding

Axis-aligned encodings are fast and often sufficient, but they do not cover general polyhedral regions (for example, regions defined by non-axis-aligned linear inequalities). The polyhedral backend in `PLPolyhedra.jl` supports upsets and downsets given as finite unions of rational polyhedra.

Polyhedral feasibility viewpoint. In principle, one could build a full arrangement of all supporting hyperplanes and enumerate its cells, but this can be expensive. The implementation instead follows a feasibility-driven approach: it enumerates candidate signatures and checks whether the region defined by that signature is nonempty.

Concretely, for each candidate signature $(y, z) \in \{0,1\}^{m+r}$, one forms the set

$$R(y,z) := \Big(\bigcap_{i:y_i=1} U_i\Big) \cap \Big(\bigcap_{i:y_i=0} (Q \setminus U_i)\Big) \cap \Big(\bigcap_{j:z_j=0} D_j\Big) \cap \Big(\bigcap_{j:z_j=1} (Q \setminus D_j)\Big),$$

and asks whether $R(y,z)$ is nonempty. Because the U_i and D_j are unions of polyhedra, each complement is also a union of polyhedra and halfspace-defined pieces, so this feasibility test branches over choices of pieces. Branches that yield an empty intersection are discarded.

Exact arithmetic for correctness. Feasibility and emptiness checks are performed using rational polyhedral computations (via `Polyhedra.jl` and `CDDLib.jl` in the implementation). This is aligned with the "exactness versus speed" discussion in Section 2.7: for encoding construction, correctness of region adjacency and signature assignment is structurally important, so exact arithmetic is often the right default.

Output and point-location implications. The polyhedral backend still outputs a signature poset P (or an explicit finite poset) together with an encoding map π. The difference is that π now needs a polyhedral point-location routine. TAMER-OP stores each realized region together with a witness point and uses a two-stage membership test (fast floating-point rejection followed by exact verification near boundaries) to implement π efficiently; see Subsection 6.5.

6.4 Constructing the finite poset structure

Once a finite set of realized signatures has been computed, TAMER-OP must turn it into a finite poset P and transport the input presentation to a P-module.

Signature poset versus explicit finite poset

Let $\{(y^{(a)}, z^{(a)})\}_{a=1}^{R}$ be the realized signatures, where $y^{(a)} \in \{0,1\}^m$ and $z^{(a)} \in \{0,1\}^r$. The canonical partial order used in all backends is the coordinatewise order:

$$a \leq_P b \quad \Longleftrightarrow \quad y^{(a)} \leq y^{(b)} \text{ and } z^{(a)} \leq z^{(b)} \quad \Longleftrightarrow \quad \mathrm{supp}(y^{(a)}) \subseteq \mathrm{supp}(y^{(b)}),\ \mathrm{supp}(z^{(a)}) \subseteq \mathrm{supp}(z^{(b)}).$$

This is exactly the order inherited from $\{0,1\}^{m+r}$.

In code this is realized in two interchangeable forms:

- `SignaturePoset` (used when `poset_kind = :signature`): store the bitvectors and implement $\leq_P$ by subset checks on bitvectors.
- `FinitePoset` (used when `poset_kind = :dense`): materialize the full reflexive transitive order relation as an $R \times R$ BitMatrix.

The second form is larger in memory but can accelerate repeated closure computations (upsets, downsets, cover edges), which appear frequently in later invariant computation.

Transporting indicator data to P

The transport step is the computational instantiation of Proposition 6.2. Each birth upset U_i induces an upset $\widehat{U}_i \subseteq P$ by reading off the ith birth bit in y:

$$\widehat{U}_i := \{a \in P : y_i^{(a)} = 1\}.$$

Similarly, each death downset D_j induces a downset $\widehat{D}_j \subseteq P$ by

$$\widehat{D}_j := \{a \in P : z_j^{(a)} = 0\}.$$

(Equivalently, membership in the complement $Q \setminus D_j$ is recorded by $z_j = 1$.)

These $\widehat{U}_i$ and $\widehat{D}_j$ are stored as bitsets on P in the `FiniteFringe.FringeModule` representation.

Transporting the monomial matrix

Given $\Phi \in \Bbbk^{r \times m}$ describing the map $\bigoplus_i \Bbbk[U_i] \to \bigoplus_j \Bbbk[D_j]$, we must produce a matrix $\widehat{\Phi}$ describing $\bigoplus_i \Bbbk[\widehat{U}_i] \to \bigoplus_j \Bbbk[\widehat{D}_j]$. By Lemma 29, the only structural constraint is:

$$\widehat{\Phi}_{ji} = 0 \quad \text{whenever} \quad \widehat{U}_i \cap \widehat{D}_j = \varnothing.$$

In a signature encoding this intersection test is purely combinatorial: the intersection is nonempty if and only if there exists some region $a \in P$ with $y_i^{(a)} = 1$ and $z_j^{(a)} = 0$. TAMER-OP therefore constructs $\widehat{\Phi}$ by retaining the original coefficient Φ_{ji} when the intersection is nonempty and forcing it to zero otherwise.

At a conceptual level, this step is where the encoding "forgets geometry" and retains only the finite combinatorics needed for algebraic computation.

6.5 Point location and evaluation map

The encoding map $\pi\colon Q \to P$ must be computable on demand, because later pipeline stages repeatedly query fibers $H(\pi(q))$ and structure maps along comparabilities in P. TAMER-OP therefore treats point location as part of the encoding object, not as an afterthought.

Grid-based point location (lattice and axis-aligned PL)

In the lattice and axis-aligned PL backends, Q is subdivided using coordinate thresholds, so locating a point reduces to finding which slab index it occupies along each axis.

Lattice case. A `ZnEncodingMap` stores:

- the critical coordinate lists C_k for each axis,
- the realized signature dictionary `sig_to_region`,
- optionally a lookup table `cell_to_region` when the grid is small enough to store (controlled by `max_lookup_cells`).

Given $g \in \mathbb{Z}^n$, one computes the cell index by n binary searches (`searchsortedlast`) in the sorted coordinate lists, costing $O(\sum_{k=1}^{n} \log |C_k|)$ comparisons. If `cell_to_region` is present, region lookup is $O(1)$ afterward. If it is absent, the implementation recomputes the signature key by evaluating the structured flat/injective predicates (still finite and typically small), and then looks up the region in a dictionary. This makes the time per query essentially either:

$$O(n \log K) \quad \text{(with lookup table)}, \qquad O(n(m+r)) \quad \text{(recompute signature)},$$

where $K = \max_k |C_k|$.

Axis-aligned PL case. A `BoxEncodingMap` works analogously with real thresholds. Point location again uses n binary searches in sorted coordinate lists, followed by a cell-to-region lookup.

Polyhedral point location

In the polyhedral backend, the region fibers are not axis-aligned boxes but rational polyhedra (or finite unions thereof). A naive point location routine would test membership against every region, which is worst-case $O(R)$ region tests per query, with each region test involving $O(Hn)$ inequality evaluations for H halfspaces. TAMER-OP implements a more structured routine in `PLPolyhedra.jl`:

- each realized region stores a witness point and a floating-point approximation of its inequalities;
- queries are first filtered using cheap floating-point inequality checks that rapidly reject most regions;
- candidates that survive the float filter are verified using exact rational arithmetic near boundaries.

This does not change the worst-case asymptotics but substantially improves expected performance in regimes where regions are moderately separated or when a coarse candidate set can be obtained from coordinate heuristics.

6.6 Building a common encoding for multiple modules

Most downstream tasks in TAMER-OP are comparative: one wants to compute invariants for a *collection* of modules and then feed those outputs into statistical summaries. For this it is highly advantageous, and often necessary, that all modules be encoded over a *common* finite poset P so that their invariants are computed on a shared combinatorial domain.

Common encoding by union of generator data. Given finitely many modules $M^{(1)}, \ldots, M^{(T)}$ presented by upsets/downsets $\{U_i^{(t)}\}$ and $\{D_j^{(t)}\}$, TAMER-OP constructs a common encoding by forming the combined signature map associated to the union of all these regions. In practice, this means:

(i) concatenate the birth and death region lists across all modules,

(ii) run the chosen backend encoding algorithm once on the combined lists to obtain a common signature poset P and map $\pi\colon Q \to P$,

(iii) transport each module's presentation to P by reading off the relevant coordinates of the signature.

This strategy is implemented by functions such as `encode_from_flanges` in `ZnEncoding.jl` and `encode_from_PL_fringe` in `PLPolyhedra.jl`. The result is a single poset P that is subordinate to every module in the family.

Theoretical context. The existence of a common encoding for a finite family is built into Miller's framework at the level of constant subdivisions, but subtle issues arise if one asks for common encodings that capture *all* homomorphisms (full subcategory versus tame morphisms). Waas gives sufficient

geometric connectivity conditions under which encodable modules form a full abelian subcategory and common encodings capture all morphisms [Waa24]. TAMER-OP's stance is more pragmatic: for a fixed finite set of modules and a fixed set of operations (the invariants computed later), a common encoding built from the union of the modules' generator regions is the appropriate computational object.

6.7 Complexity and failure modes

Encoding is the stage where combinatorics can blow up. It is therefore the right place to state explicit complexity bounds and to clarify what kinds of inputs are expected to be tractable.

Complexity of grid-based encodings

Consider either the lattice or axis-aligned PL backend. Let n be the parameter dimension, let m and r be the number of birth and death regions in the presentation, and let C_k be the critical coordinate set on axis k. Define

$$K_k := |C_k|, \qquad N_{\text{cell}} := \prod_{k=1}^{n} (K_k + 1), \qquad R := |\text{im}(\sigma)| \leq N_{\text{cell}}.$$

Enumerating signatures. In both backends, the encoding algorithm visits each cell once and performs constant work plus updates for constraint events crossing cell boundaries. A clean worst-case bound that is easy to compare across variants is:

$$T_{\text{sig}} = O((m + r)\, n\, N_{\text{cell}})\,,$$

which is the cost of evaluating all membership predicates per cell in a naive scheme. The incremental scan improves constants by replacing full reevaluation with sparse updates. Its asymptotic upper bound can be phrased as

$$T_{\text{scan}} = O(N_{\text{cell}} + E_{\text{flip}})\,,$$

where E_{flip} is the total number of bit flips processed across the scan. In the worst case $E_{\text{flip}} = O(n(m + r)\, N_{\text{cell}})$ because each generator bit can flip at most once per axis per fiber of the other coordinates. Thus both descriptions agree at the big-O level.

Building the poset order. Given R realized signatures, materializing the order relation by pairwise subset tests costs

$$T_{\leq} = O\left(R^2 \cdot \frac{m + r}{w}\right),$$

where w is the machine word size used by the bitset representation (typically 64). If one also computes the Hasse cover relation from the full order matrix (as in `FiniteFringe._compute_cover_edges_bitset`), the worst-case bound is

$$T_{\text{cover}} = O\left(\frac{R^3}{w}\right),$$

though in practice this is mitigated by bit-parallel operations and by caching cover edges only when they are actually requested downstream.

Memory. Storing `cell_to_region` requires $O(N_{\text{cell}})$ memory, which is precisely why `ZnEncodingMap` makes it optional (via `max_lookup_cells`). Storing the signature bitvectors requires $O(R(m + r)/w)$ memory. Materializing the full order matrix requires $O(R^2/w)$ memory.

Interpreting the bounds. These formulas highlight the central design tradeoff: for fixed parameter dimension n, the grid size N_{cell} is polynomial in the number of critical coordinates per axis. In the lattice backend, each flat or injective contributes at most one critical coordinate per constrained axis, so a crude but informative bound is $K_k = O(m + r)$, hence

$$N_{\text{cell}} = O((m + r)^n) \quad \text{(for fixed } n\text{)}.$$

This is the main sense in which finite encodings make computation feasible in the common $n = 2$ or $n = 3$ regimes: the potentially infinite parameter domain is compressed to a finite poset whose size grows polynomially in the presentation size when n is small. The same expression also makes the failure mode explicit: for large n, $(m + r)^n$ is prohibitive, and one must use more problem-specific reductions.

Complexity of the polyhedral feasibility backend

For the polyhedral backend, a different parameterization is appropriate. Let H be the total number of halfspaces needed to describe all polyhedral pieces in all U_i and D_j, and let B measure the branching factor induced by unions and complements (formally, the number of piece-selection branches explored).

Feasibility enumeration. The implementation enumerates candidate signatures and branches over polyhedral pieces, performing an emptiness check for each branch. A coarse upper bound is

$$T_{\text{poly}} = O\big(2^{m+r} \cdot B \cdot T_{\text{feas}}(n, H)\big)\,,$$

where $T_{\text{feas}}(n, H)$ is the cost of a rational polyhedral feasibility or emptiness test in dimension n with H inequalities. This is exponential in the number of generators $m+r$, and this backend is therefore intended for regimes where $m + r$ is small but each region has non-axis-aligned geometry. The implementation includes a hard cap `max_regions` to prevent runaway enumeration.

Typical failure modes

The most common failure modes are not subtle.

- **Combinatorial explosion in N_{cell} or R.** This is the dominant risk in grid-based encodings and is intrinsic to the problem. TAMER-OP mitigates it by offering signature-based collapse (often $R \ll N_{\text{cell}}$) and by limiting lookup-table storage.
- **Exponential branching in polyhedral unions.** This is intrinsic to complement and union handling in the feasibility backend; the mitigation is to use it only when $m + r$ and the number of pieces per region are small, and to cap `max_regions`.
- **Boundary ambiguity in $\mathbb{R}^n$.** The axis-aligned backend avoids this by sampling interior points via `strict_eps`. The polyhedral backend avoids it by using exact arithmetic for final membership checks.

6.8 Correctness and diagnostics

The encoding constructors are designed so that correctness follows from a small number of verifiable invariants: signatures are computed on cells where they are constant, and the poset and transported presentation are defined by those signatures.

Proposition (Correctness criterion for signature encodings)**.** *Suppose Q is partitioned into subsets (cells) $\{C_\alpha\}$ such that for every i and j, the predicates $q \in U_i$ and $q \in D_j$ are constant on each C_α. Let P be the set of realized signatures on the cells, ordered coordinatewise, and let $\pi\colon Q \to P$ send q to the signature of its cell. Then π is an encoding map subordinate to every indicator module $\Bbbk[U_i]$ and $\Bbbk[D_j]$, and any fringe presentation built from these indicators transports to a P-presentation whose pullback recovers the original one.*

Proof. By hypothesis, each U_i is a union of cells and each D_j is a union of cells. Thus U_i and D_j are unions of fibers of π, hence they are inverse images of subsets of P. The transported subsets are exactly $\widehat{U}_i$ and $\widehat{D}_j$ from Subsection 6.4, so $\Bbbk[U_i] \cong \pi^*\Bbbk[\widehat{U}_i]$ and similarly for $\Bbbk[D_j]$. The statement for a fringe presentation then follows from functoriality: the stalkwise matrix description of the map is respected on each fiber because all indicator data are constant there. □

Diagnostics in practice. TAMER-OP exposes and internally relies on a collection of sanity checks that align with Proposition 6.8:

- **Witness-based verification.** Each realized signature region is stored with at least one representative (cell representative in grid backends, polyhedral witness point in the feasibility backend). This makes it straightforward to check that the recorded signature agrees with direct membership evaluation.
- **Monotonicity checks.** Because the intended encoding map is order-preserving (Lemma 41), violations of monotonicity indicate either an implementation bug or a boundary-handling issue in $\mathbb{R}^n$; the `strict_eps` and exact arithmetic mechanisms are designed specifically to prevent such issues.
- **Size and cap reporting.** Encoding constructors record the numbers N_{cell} and R (or directly R in the feasibility backend) and compare them against user-specified caps such as `max_regions`. When a cap is hit, the encoding is intentionally aborted rather than silently producing an incomplete poset.

At the thesis level, the main conclusion is that the encoding step is both the mathematically essential and the algorithmically delicate part of the TAMER-OP pipeline. It is where tameness is converted into finite combinatorics, and it is also where the ambient dimension and the geometry of the birth/death regions determine feasibility. The next sections treat invariants and derived constructions under the assumption that this finite core object (P, H) has been successfully constructed.

7 Invariants

7.1 Meta principle: invariants factor through the encoding

7.2 Dimension-type invariants

7.3 Rank-type invariants

7.4 Mobius inversions and signed measures

7.5 Slice-based invariants and distances

7.6 Vectorizations and kernels

7.7 Homological invariants derived from resolutions

7.8 Geometry-aware invariants for when $Q = \mathbb{R}^n$

7.9 Stability and robustness discussion

7.10 Practical API summary

8 Homological Algebra and Derived Functors

8.1 The Abelian Category of P-modules

8.2 Projective and Injective Objects

8.3 Resolutions

8.4 Ext and Tor: definitions and computational model

8.5 Multiplicative structures and higher structure

8.6 Double complexes and spectral sequences

8.7 The ergonomics of derived categories

8.8 Relationship to multigraded commutative algebra

8.9 What you use all this for in TDA

9 Transport and comparison

9.1 Change of posets as functorial transport

9.2 Comparing modules on different encodings

9.3 Comparison via invariants

9.4 Interoperability with multipers and RIVET

9.5 Case studies

10 Statistics

10.1 Data model for experiments

10.2 Feature extraction recipes

10.3 Basic inferential statistics

10.4 Learning-oriented workflows

10.5 Diagnostic plots

10.6 Reproducibility and reporting

11 Speed optimizations

11.1 Where time is spent

References

[Mil20] E. Miller. *Homological algebra of modules over posets.* arXiv:2008.00063, 2020.

[Waa24] L. Waas. *Notes on abelianity of categories of finitely encodable persistence modules.* arXiv:2407.08666, 2024.

[Mil23] E. Miller. *Stratifications of real vector spaces from constructible sheaves with conical microsupport. Journal of Applied and Computational Topology*, 7(3):473–489, 2023. doi:10.1007/s41468-023-00112-1.

[KS18] M. Kashiwara and P. Schapira. *Persistent homology and microlocal sheaf theory. Journal of Applied and Computational Topology*, 2(1–2):83–113, 2018. arXiv:1705.00955.

[CB15] W. Crawley-Boevey. *Decomposition of pointwise finite-dimensional persistence modules. Journal of Algebra and Its Applications*, 14(5):1550066, 2015. doi:10.1142/S0219498815500668.

[ZC05] A. Zomorodian and G. Carlsson. *Computing persistent homology. Discrete & Computational Geometry*, 33:249–274, 2005. doi:10.1007/s00454-004-1146-y.

[CZ09] G. Carlsson and A. Zomorodian. *The theory of multidimensional persistence. Discrete & Computational Geometry*, 42:71–93, 2009. doi:10.1007/s00454-009-9176-0.

[CEH07] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. *Stability of persistence diagrams. Discrete & Computational Geometry*, 37(1):103–120, 2007. doi:10.1007/s00454-006-1276-5.

[ELZ02] H. Edelsbrunner, D. Letscher, and A. Zomorodian. *Topological persistence and simplification. Discrete & Computational Geometry*, 28:511–533, 2002. doi:10.1007/s00454-002-2885-2.

[Les15] M. Lesnick. *The theory of the interleaving distance on multidimensional persistence modules. Foundations of Computational Mathematics*, 15(3):613–650, 2015. doi:10.1007/s10208-015-9255-y.

[LW15] M. Lesnick and M. Wright. *Interactive visualization of 2-D persistence modules.* arXiv:1512.00180, 2015.

[RIVET] M. Lesnick and M. Wright. *RIVET: The rank invariant visualization and exploration tool* (software). `https://github.com/rivetTDA/rivet`.

[LS24] D. Loiseaux and H. Schreiber. *multipers: Multiparameter persistence for machine learning. Journal of Open Source Software*, 9(103):6773, 2024. doi:10.21105/joss.06773.

[CDSGO16] F. Chazal, V. de Silva, M. Glisse, and S. Y. Oudot. *The Structure and Stability of Persistence Modules.* SpringerBriefs in Mathematics. Springer, 2016. doi:10.1007/978-3-319-42545-0.

[Oud15] S. Y. Oudot. *Persistence Theory: From Quiver Representations to Data Analysis.* Mathematical Surveys and Monographs, Vol. 209. American Mathematical Society, 2015.

[EH10] H. Edelsbrunner and J. L. Harer. *Computational Topology: An Introduction.* American Mathematical Society, 2010.

[Mac98] S. Mac Lane. *Categories for the Working Mathematician.* Graduate Texts in Mathematics, Vol. 5. Springer, 2nd edition, 1998. doi:10.1007/978-1-4757-4721-8.

[Wei94] C. A. Weibel. *An Introduction to Homological Algebra.* Cambridge Studies in Advanced Mathematics, Vol. 38. Cambridge University Press, 1994.

[Eis95] D. Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry.* Graduate Texts in Mathematics, Vol. 150. Springer, 1995. doi:10.1007/978-1-4612-5350-1.

[MS05] E. Miller and B. Sturmfels. *Combinatorial Commutative Algebra.* Graduate Texts in Mathematics, Vol. 227. Springer, 2005. doi:10.1007/b138602.